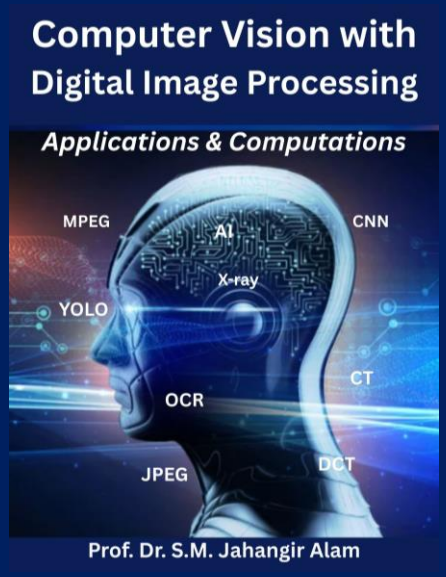
Lecture 1.1

Course Name: Digital Image Processing
Course code: 4245

Image Acquisition & Human Visual System: Knowledge of the Human Eye's Structure and how it Perceives Images.



Instructor Name:

Professor Dr. Jahangir Alam

Professor, Dept. of CSE, BAUST

Image Acquisition



- Camera/CCD camera
- Flat bed scanner

charge-coupled device (CCD) or complementary metal–oxide–semiconductor (CMOS) image sensors

Image Acquisition & Converted into Digital Form

Example:

- Suppose ADC uses 8 bits → 256 levels (0 to 255).
- A pixel voltage range is 0V to 2.56V.
- Convert voltage to digital value:

$$Digital\ Value = Voltage \times \frac{255}{2.56}$$

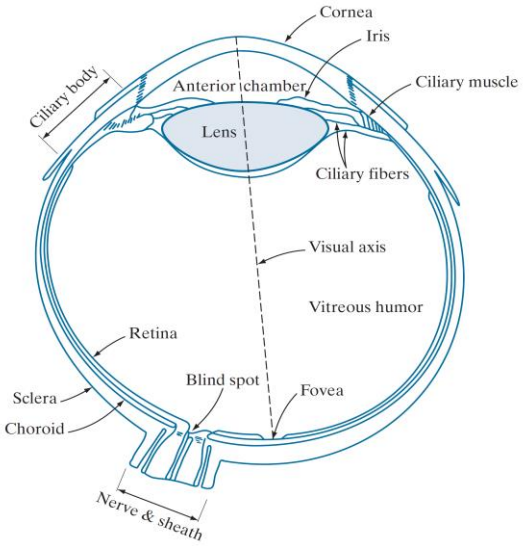
- For Pixel #1:

$$1.2V \times \frac{255}{2.56} \approx 119$$

- So, pixel #1's digital value = 119 (on scale 0–255).

Structure of the Human Eye

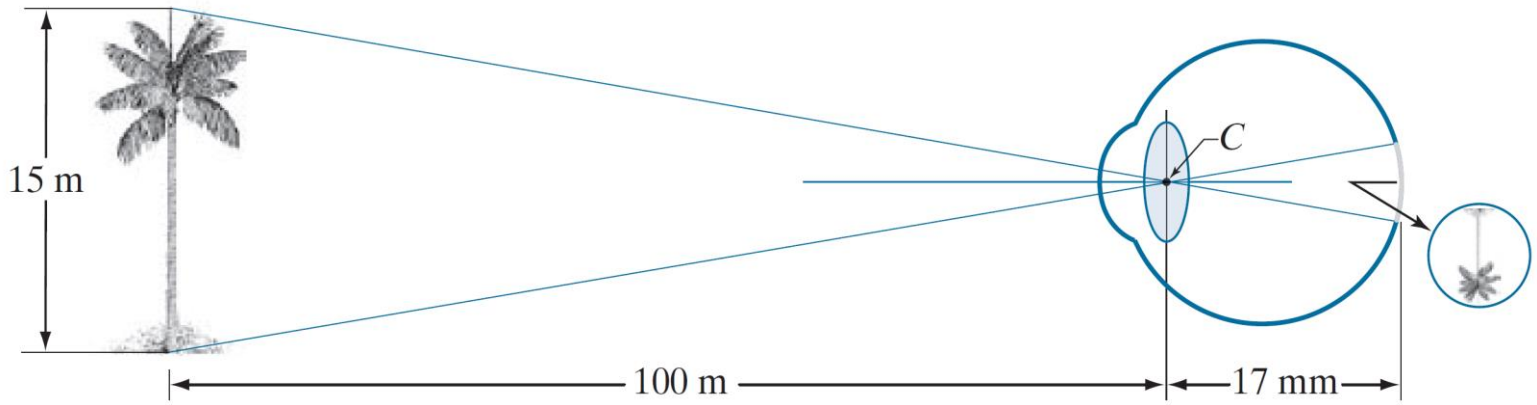
- Light enters the eye through a **small hole** called the **pupil**,
- A black looking aperture having the quality of contraction of eye when exposed to bright light and is focused on the **retina** which is like a camera film.
-
- The lens, iris, and cornea are nourished by clear fluid, know as anterior chamber.
- The fluid flows from ciliary body to the pupil and is absorbed through the channels in the angle of the anterior chamber.



Structure of the Human Eye

- Cones in eye number between 6 to 7 million which are highly sensitive to colors.
- Human visualizes the colored image in daylight due to these cones.
- The cone vision is also called as photopic or bright-light vision.
- Rods in the eye are much larger between 75 to 150 million and are distributed over the retinal surface.
- Rods are not involved in the color vision and are sensitive to low levels of illumination.

Image Formation in the Eye



1. Image is formed in retina.

2. The distance between the lens and the retina is about 17mm and the focal length is approx. 14mm to 17mm.

Human Eye's Structure and Perceives Images

Step 1: Light Enters the Eye

Step 2: Pupil and Iris Control Light Amount

Step 3: Lens Focuses the Image

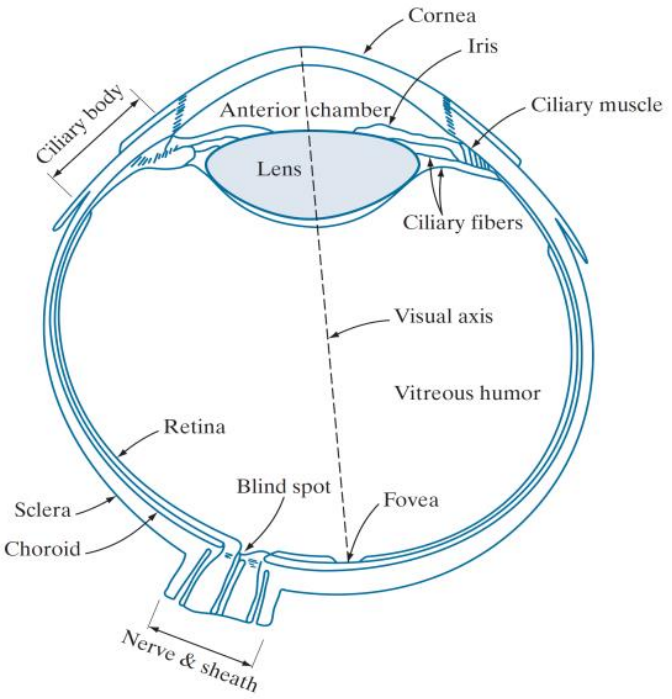
Step 4: Image Formation on Retina

Step 5: Photoreceptors Detect Light.

Step 6: Signal Conversion

Step 7: Signal Transmission to Brain

Step 8: Brain Processes the Image



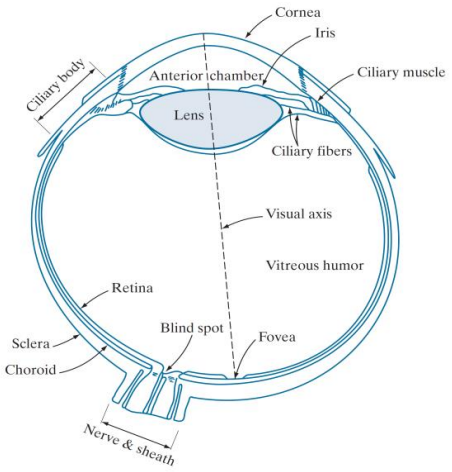
Visual Perception

- **Visual perception** is the process through our **brain interprets visual signals** received from the eyes. It includes perception of:
    - Shape
    - Size
    - Color
    - Depth
    - Motion
    - Patterns

Pupil Detection or Gaze Estimation

Pupil detection:

pupil appears darker than the surrounding iris/sclera.



Frame 1:						
200	200	190	180	190	200	200
200	100	50	30	50	100	200
190	50	20	10	20	50	190
180	40	10	0	10	40	180
190	50	20	10	20	50	190
200	100	50	30	50	100	200
200	200	190	180	190	200	200

Pupil Detection or Gaze Estimation

If pixel intensity < **30**, mark it as part of the **pupil**

Binary Mask .

	0	0	0	0	0	0	0
	0	0	0	**1**	0	0	0
M=	0	0	**1**	**1**	**1**	0	0
	0	0	**1**	**1**	**1**	0	0
	0	0	**1**	**1**	**1**	0	0
	0	0	0	**1**	0	0	0
	0	0	0	0	0	0	0

(Center of Detected Pupil)

Collect all **(row, col)** positions where mask = 1:

(1,3), (2,2), (2,3), (2,4), (3,2), (3,3), (3,4), (4,2), (4,3), (4,4), (5,3)

Avg. row: (1+2+2+2+3+3+3+4+4+4+5)/11 = 33 / 11 = **3.0**

Avg. col: (3+2+3+4+2+3+4+2+3+4+3)/11 = 33 / 11 = **3.0**

Pupil center = (3, 3)

Pupil Detection or Gaze Estimation

Moved **right** in the next frame:
Pupil center: (3, 4)

0	0	0	0	0	0	0
0	0	0	**1**	0	0	0
0	0	**1**	**1**	**1**	0	0
0	0	**1**	**1**	**1**	0	0
0	0	**1**	**1**	**1**	0	0
0	0	0	**1**	0	0	0
0	0	0	0	0	0	0

$$\boldsymbol{Gaze\ Vector}$$
$$= Pupil\ center - Eye\ center$$
$$= (3,4) - (3,3) = (0,+1)$$

So, the person is looking slightly to **right**.

Normalize Gaze Direction

- Eye width = 7 columns
- $\Delta x = 1 \rightarrow gaze_x = 1 / (7/2) = \boldsymbol{\sim 0.29\ right}$

Pupil Detection or Gaze Estimation

A **gaze vector** is a mathematical representation of **where a person is looking**. It's usually modeled as a **3D direction vector** that starts from the eye (or eye center) and points toward the gaze direction.

The gaze vector is

$$g = \begin{bmatrix} cos\emptyset\ sin\theta \\ sin\emptyset \\ cos\emptyset cos\theta \end{bmatrix}$$

This is very common in **computer vision and eye-tracking**.

Classification of Gaze Estimation

Gaze Vector	Direction
(0, 0)	Center
(0, +1)	Right
(0, –1)	Left
(+1, 0)	Down
(–1, 0)	Up

Gaze detection using CNN GazeNet

Input:

- **Face image**
- **Left eye image**
- **Right eye image**
- (Optionally) **head pose**

And predicts:

- A **3D gaze direction vector** (usually (x, y, z)) or gaze angles (yaw, pitch).

Gaze detection using CNN GazeNet

Let's use small numerical vectors to represent the **image features** (after CNN encoding):

Left Eye Features: $eL = [0.5, 0.2]$

Right Eye Features: $eR = [0.6, 0.1]$

Face Features: $f = [0.4, 0.3]$

Concatenate Features

$$x = [eL, eR, f] = [0.5, 0.2, 0.6, 0.1, 0.4, 0.3]$$

Thus, total input vector size = 6.

Gaze detection using CNN GazeNet

Fully Connected Layer (FC1)

Let's define:

- **Weight matrix** $W \in R^{2\times 6}$
- **Bias** $b \in R^{2}$

Let:

$$W = [0.2, 0.1, 0.1, 0.1, 0.3, 0.2, 0.1, 0.2, 0.3, 0.2, 0.1, 0.1],$$

$$b = [0.05 \quad 0.1]$$

Then:

$$z = W \cdot x + b$$

$$x = [0.5, 0.2, 0.6, 0.1, 0.4, 0.3]$$

$$z1 = (0.2)(0.5) + (0.1)(0.2) + (0.1)(0.6) + (0.1)(0.1) + (0.3)(0.4) + (0.2)(0.3) + 0.05 = 0.42$$

Gaze detection using CNN GazeNet

$$W = [0.2, 0.1, 0.1, 0.1, 0.3, 0.2, 0.1, 0.2, 0.3, 0.2, 0.1, 0.1]$$

$$x = [0.5, 0.2, 0.6, 0.1, 0.4, 0.3]$$

$$z2$$
$$= (0.1)(0.5) + (0.2)(0.2) + (0.3)(0.6) + (0.2)(0.1)$$
$$+ (0.1)(0.4) + (0.1)(0.3) + 0.1$$
$$= 0.05 + 0.04 + 0.18 + 0.02 + 0.04 + 0.03 + 0.1 = 0.46$$

$$tanh(0.42) \approx 0.40, tanh(0.46) \approx 0.43$$

Therefore:

$$Predicted\ Gaze\ Vector = (0.40, 0.43)$$

Predicted 2D gaze direction (simplified):

$$(x, y) = (0.40, 0.43)$$

This would correspond to **a gaze pointing slightly up and to the right**.

Images and pictures

- **Picture**: A visual representation made for humans to see (photo, drawing, painting).
- **Image**: meaning a **digital representation of visual data**.
- An **image** is a single picture which represents something.

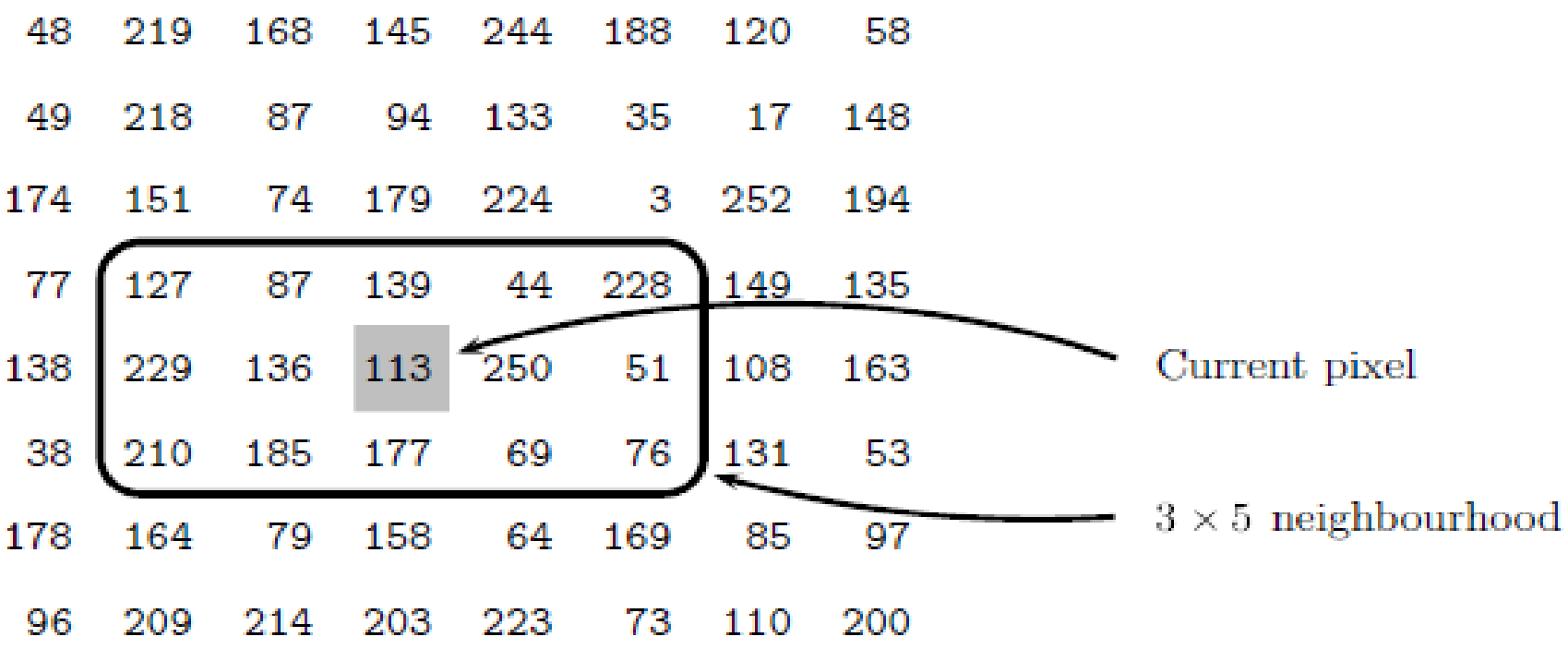
Key differences (easy table)

- It may be a **picture** of a person, of people or animals, or of an outdoor scene, or a microphotograph of an electronic component, or the result of medical imaging.
- Image as being a two-dimensional function x and y which takes any continuous values;
- A digital image takes all discrete values;

Images and Digital Images

- A large array of discrete dots.
- Dots are called picture elements, or more simply pixels.
- The pixels surrounding a given pixel constitute its neighborhood.

Neighborhood is a rectangular block.



Elements of Digital Image Processing

20	30	35	25
50	60	55	40
90	100	95	80
150	160	140	120

Pixels and Resolution calculation

- Each pixel is stored as 0–255.
- Grayscale values for the 4×4 image:

If the bottom-right pixel is 120 in grayscale:

- Normalized value = $120/255 \approx 0.47$
- This is what algorithms use for edge detection.

1 inch = 2.54 cm

The, 4 pels/cm → $4 \times 2.54 \approx 10.16$ pels/inch≈ 10 PPI

So, the scanned image:

- Width in pixels = 2 cm×4 pixels/cm=8 pixels
- Height in pixels = 8 pixels

- Resolution in PPI = 10 PPI

Pixels & Resolution

Images

This is a Coursera Course on the "Fundamentals of Digital Image and Video Processing"

1 bit per pixel



8 bits per pixel



24 bits per pixel

Pixels & Resolution

- **Resolution (NxM):** N pixels height × M pixels wide.

- Photo resolution is calculated by **NxM**.

- MP is a million pixels. "640 × 480 display".
- 640 × 480 = 307,200 pixels, or 0.3 megapixels.

- GP image is a digital image bitmap composed of one billion (10^9) pixels (picture elements).

- TP image composed of one trillion (10^{12}) pixels.

- Size = 10120 x 6328 = 64,039,360 pixels, Pixel count = 64.04 MP pixels.

Measuring spatial resolution

- Dots per inch: Dots per inch or DPI is usually used in monitors.
- Lines per inch: Lines per inch or LPI is usually used in laser printers.
- Pixels per inch: Pixel per inch or PPI is measure for different devices such as tablets , Mobile phones, etc.

FPS

Frames per second (FPS) is a measure of how many individual images (frames) a system displays, captures, or processes in **1s**.

- **In video**: Higher FPS means smoother motion.
  - 24 FPS → cinematic movies (slightly less smooth, but artistic look)
  - 30 FPS → common for TV broadcasts
  - 60 FPS → smooth, good for sports/gaming
  - 120 FPS+ → high-speed video, slow motion
- **In cameras**: FPS tells how many pictures per second the camera sensor can capture.
- **In games**: FPS measures how fast your GPU renders frames. Higher FPS = smoother gameplay.

FPS

- **Formula**

$$FPS = \frac{Number\ of\ frames}{Time\ in\ Seconds}$$

- Example: If a video has 300 frames and lasts 10 seconds:

$$FPS = \frac{300}{10} = 30\ fps$$

Impact of FPS:

- **Low FPS** (<20) → choppy video, lag in games.
- **Moderate FPS** (24–30) → watchable video.
- **High FPS** (60+) → smoother, more responsive, especially important in gaming and sports tracking.

Example

If you have 1080 px vertically and 1920 pixels horizontally. Then the size is 1920 px by 1080 px. If you want to check in inches then you have to check the image resolution. If the resolution is 300 dpi then that means we have 300 pixels in every inch. That means

- 1 inch =300pixels
- Also if the image size is 1920 ×1080px.
- Then the width of the image in inches will be

  1920/300 = 6.4 inches= 6.4″
- And height will be 1080/300 = 3.6 inches= 3.6″
- That means the size of image in inches will be 6.4″ × 3.6″.

Example

Calculate the size of an RGB image of size 128 × 64.

- If you are using standard 8 bits (1 byte) per color component, computing the size in bytes would be:
- 128 x 64 x 3 x 1 Byte = 24,576 bytes (3 because RGB has 3 components)
- Or, 128 x 64 x 24 bits/8= 24,576 Bytes

- 1024 Byte(B) =1 KB: 24,576 / 1024 = 24 kilobytes (KB)

- 24 / 1024 = 0.0234375 megabytes (MB)

How do you calculate the image size from resolution?

- Suppose an image is: Resolution: **1920 × 1080** pixels

Color depth: **24 bits per pixel** (8 bits per RGB channel)

- **Step 1 — Total pixels**

  1920×1080=2,073,600 pixels1920

- **Step 2 — Multiply by bits per pixel**

  2,073,600×24=49,766,400 bits

- **Step 3 — Convert bits to bytes**

  49,766,400÷8=6,220,800 bytes

- **Step 4 — Convert bytes to MB**

  6,220,800÷(1024×1024)≈5.93 MB

So, the raw image size ≈ 5.93 MB

Sources for Images

- Electromagnetic (EM) energy spectrum
- Acoustic
- Ultrasonic
- Electronic
- Synthetic images produced by computer

Examples: Visual and Infrared Imaging

Band No.	Name	Wavelength (μm)	Characteristics and Uses
1	Visible blue	0.45–0.52	Maximum water penetration
2	Visible green	0.53–0.61	Measures plant vigor
3	Visible red	0.63–0.69	Vegetation discrimination
4	Near infrared	0.78–0.90	Biomass and shoreline mapping
5	Middle infrared	1.55–1.75	Moisture content: soil/vegetation
6	Thermal infrared	10.4–12.5	Soil moisture; thermal mapping
7	Short-wave infrared	2.09–2.35	Mineral mapping

Image Perception

1. capturing the image with the eye,

2. recognizing and interpreting the image with the visual cortex in the brain.

3. Observed intensities vary as to the background.

4. observe non-existent intensities as bars in continuously varying grey levels



Image Perception of Human Perceive Contrast

- **Center pixel** (200) is much brighter than its neighbors (50 and 80).
- The human eye perceives brightness relative to surrounding pixels.

$$I = \begin{bmatrix} 50 & 80 & 50 \\ 80 & \mathbf{200} & 80 \\ 50 & 80 & 50 \end{bmatrix}$$

Average of neighbors: $(50 + 80 + 50 + 80 + 80 + 50 + 80 + 50)/8 = 520/8 = 65$

Contrast: $C = 200 - 65 = 135$

High contrast → center pixel stands out strongly.

Image Perception of Human Perceive Contrast

Pixel Position	Intensity	Neighbor Avg	Contrast I_{center} – Neighbor Avg
Center (2,2)	200	65	135
Top-left (1,1)	50	(neighbors avg ≈ 72)	-22

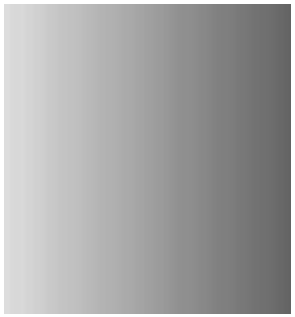
- **Image perception** relies on relative intensity, not absolute.

- Contrast helps detect edges, objects, or features in an image.

$$I = \begin{bmatrix} 50 & 80 & 50 \\ 80 & \mathbf{200} & 80 \\ 50 & 80 & 50 \end{bmatrix}$$

Image Negative

$$Image = \begin{bmatrix} 100 & 120 & 130 \\ 90 & 110 & 140 \\ 80 & 100 & 120 \end{bmatrix}$$



Apply Negative Transformation:

Negative Pixel=255−Original Pixel

$$Image = \begin{bmatrix} 255-100 & 255-120 & 255-130 \\ 255-90 & 255-110 & 255-140 \\ 255-80 & 255-100 & 255-120 \end{bmatrix} = \begin{bmatrix} 155 & 135 & 125 \\ 165 & 145 & 115 \\ 175 & 155 & 135 \end{bmatrix}$$

Negative image

Bit Depth (Concept):

If the image is 8-bit:

- Each pixel = 1 byte (8 bits)
- Values range = $2^8 = 256$ levels (0–255)

For a 3×3 image, total size = $3 \times 3 = 9$ pixels = **9 bytes** in raw form (uncompressed).

Lecture 1.2

Course Name: Digital Image Processing
Course code: 4245

Digital Image Fundamentals

Instructor Name:
Professor Dr. Jahangir Alam
Professor, Dept. of CSE, BAUST

Image Processing?

1. Improve its pictorial information for human interpretation,

2. Render it more suitable for autonomous machine perception.

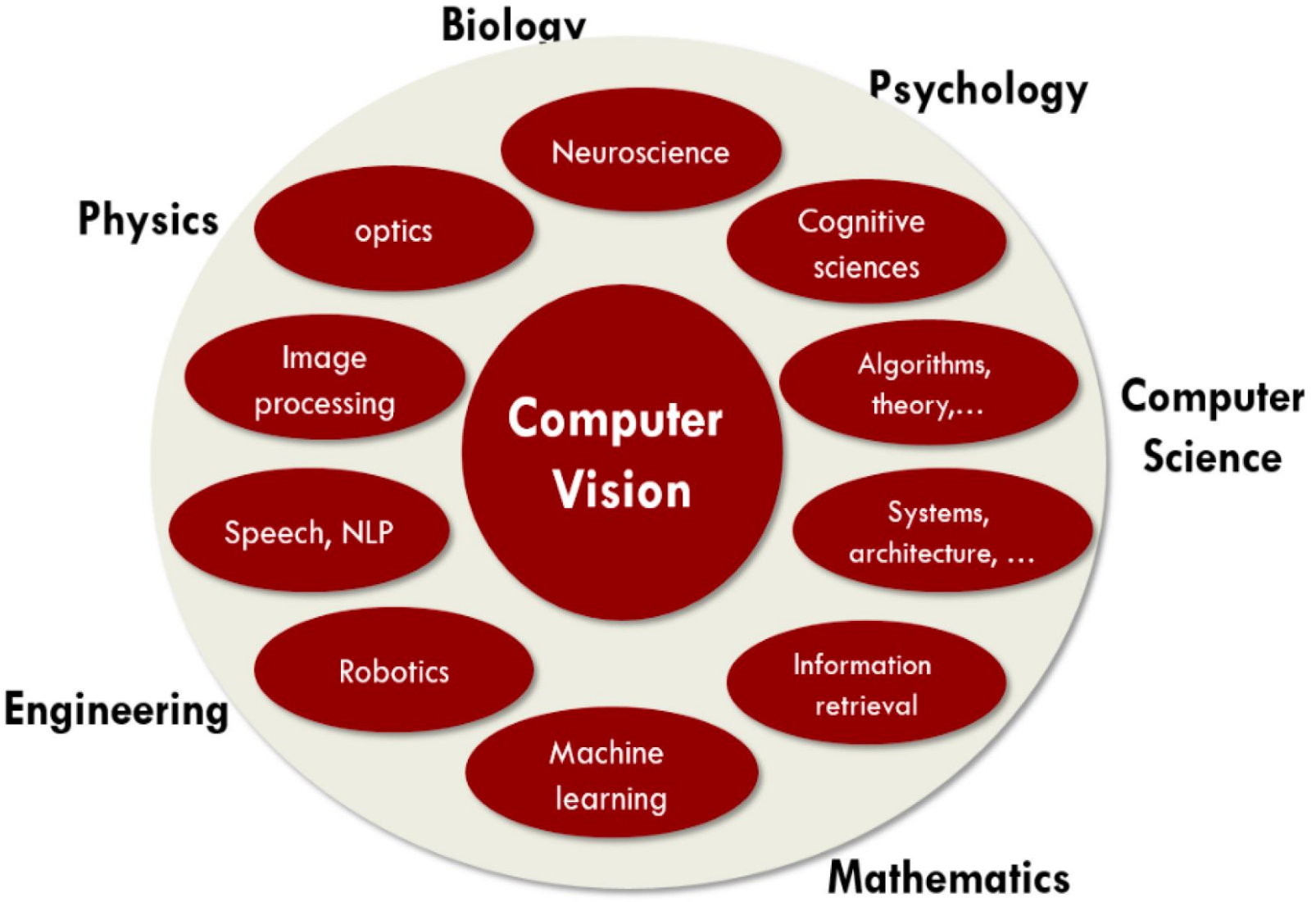
Image Analysis & Computer Vision

- Computer vision (CV)/machine vision/human vision play the single most important role in human perception.

- CV whose ultimate goal is to use computers to emulate human vision to emulate human intelligence in AI;

- The area of image analysis (also called *image understanding*) is in between image processing and computer vision.

Computer Vision



Cognitive Science

- **Cognitive Science** is the **interdisciplinary study of the mind and intelligence**.
- It explores **how humans (and machines)** acquire, represent, and use knowledge.
- Cognitive Science = *the science of the mind*, combining psychology, AI, neuroscience, linguistics, philosophy, and anthropology to explain **how we perceive, learn, remember, reason, and communicate**.

Applications of DIP

- Image sharpening and restoration
- Medical field
- Remote sensing
- Transmission and encoding
- Machine/Robot vision
- Color processing
- Pattern recognition
- Video processing
- Microscopic Imaging
- Others

Applications

1. Medical

- Inspection and interpretation of images obtained from X-rays, MRI or CT/CAT scans,
- Analysis of cell images, of chromosome karyotypes.
- MRI scan is a painless test that produces very clear images of the organs and structures inside your body (Tumors, Inflammation, Infection, Blood supply to certain organs, Blood vessels).
- MRI creates clear images inside your body using a large magnet, radio waves and a computer.

- Computed tomography (CT) uses X-rays & computers.
- MRI scans instead of CT scans to look at the non-bony parts or soft tissues inside human body.
- MRI scans are safer because don't use the damaging ionizing radiation of X-rays.



Applications

2. Agriculture
   - Satellite/aerial views of land,
   - Investigate suitability of different regions for different crops,
   - Inspection of fruit and vegetables distinguishing.

3. Industry
   - Automatic inspection of items on a production line,
   - inspection of paper samples.

4. Law enforcement
   - Fingerprint analysis,
   - sharpening or de-blurring of speed-camera images.

Purpose of Image processing

1. **Visualization:**

2. **Image sharpening and restoration:**

3. **Image retrieval:** An image of interest can be seen

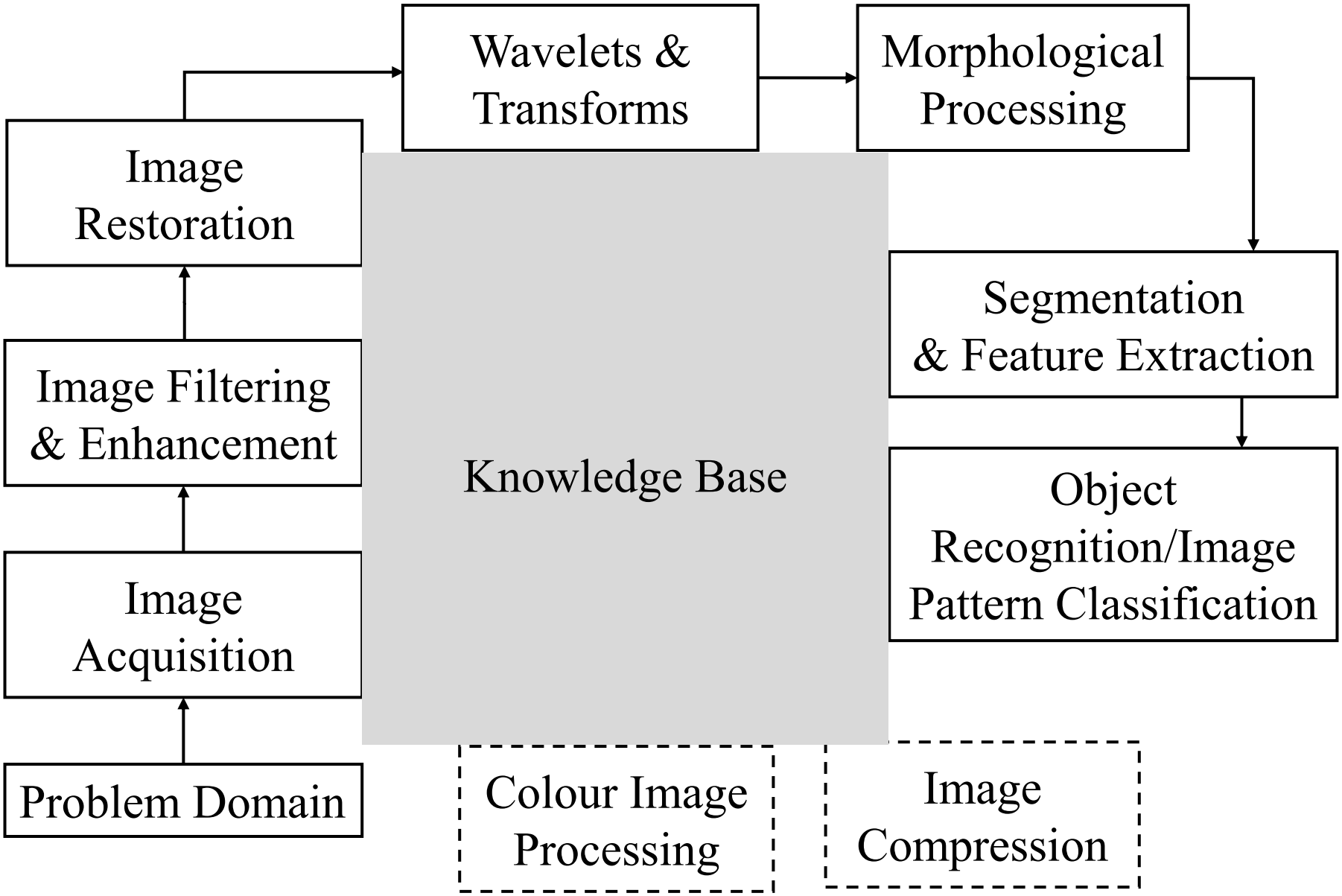
4. **Measurement of pattern:** all the objects are measured.

5. **Image Recognition:** Each object in an image can be distinguished.

Key Points

- **Image restoration:** degradation & restoring to original appearance.
- **Image enhancement:** improving an image visually.
- **Image compression:** reducing the amount of data of an image.
- **Image analysis:** the examination of image data to solve an image processing problem.
- **Image segmentation:** used to find higher level objects.
- **Feature extraction:** acquiring higher level information, such as shape or color of objects.
- **Image transforms:** used in feature extraction to find spatial frequency information.
- **Pattern classification:** used for identifying objects in an image.

Key Stages in DIP



Key Stages in DIP

Problem Domain

Image Acquisition

Image Filtering & Enhancement

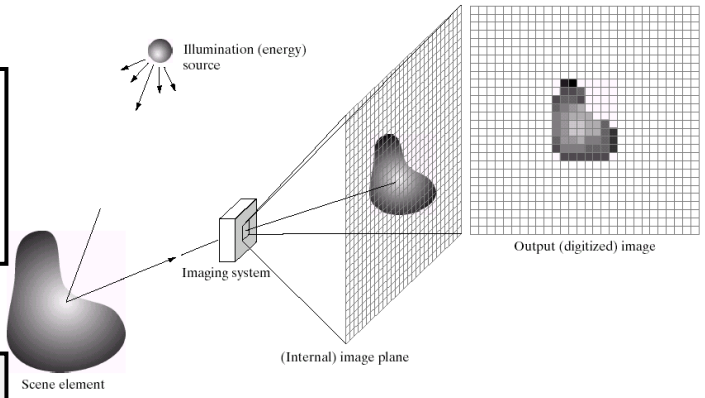
Image Restoration

Wavelets & Transforms

Morphological Processing

Segmentation & Feature Extraction

Object Recognition/Image Pattern Classification



preprocessing, such as scaling

Colour Image Processing

Image Compression

Key Stages in DIP

Problem Domain

Image Acquisition

Image Filtering & Enhancement

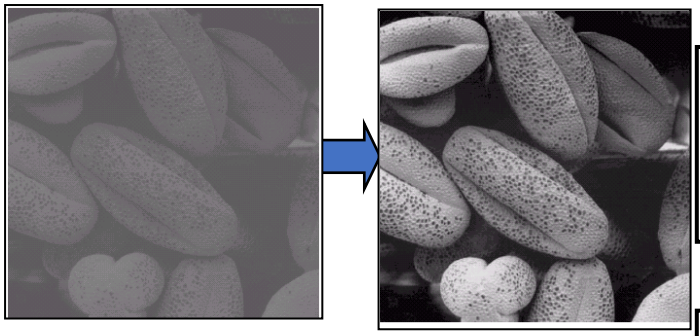
Image Restoration

Wavelets & Transforms

Morphological Processing

Segmentation & Feature Extraction

Object Recognition/Image Pattern Classification

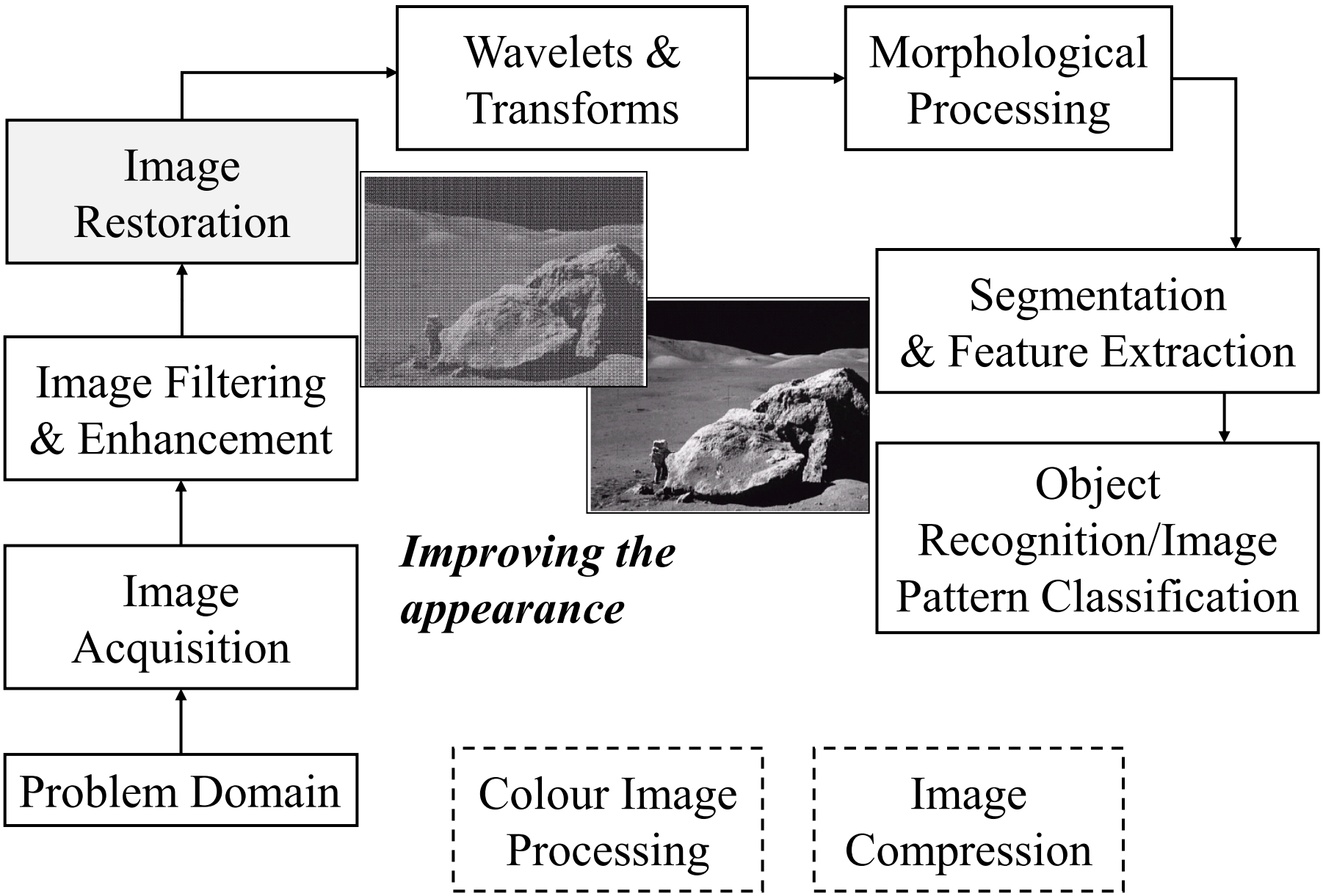


Result is more suitable than the original

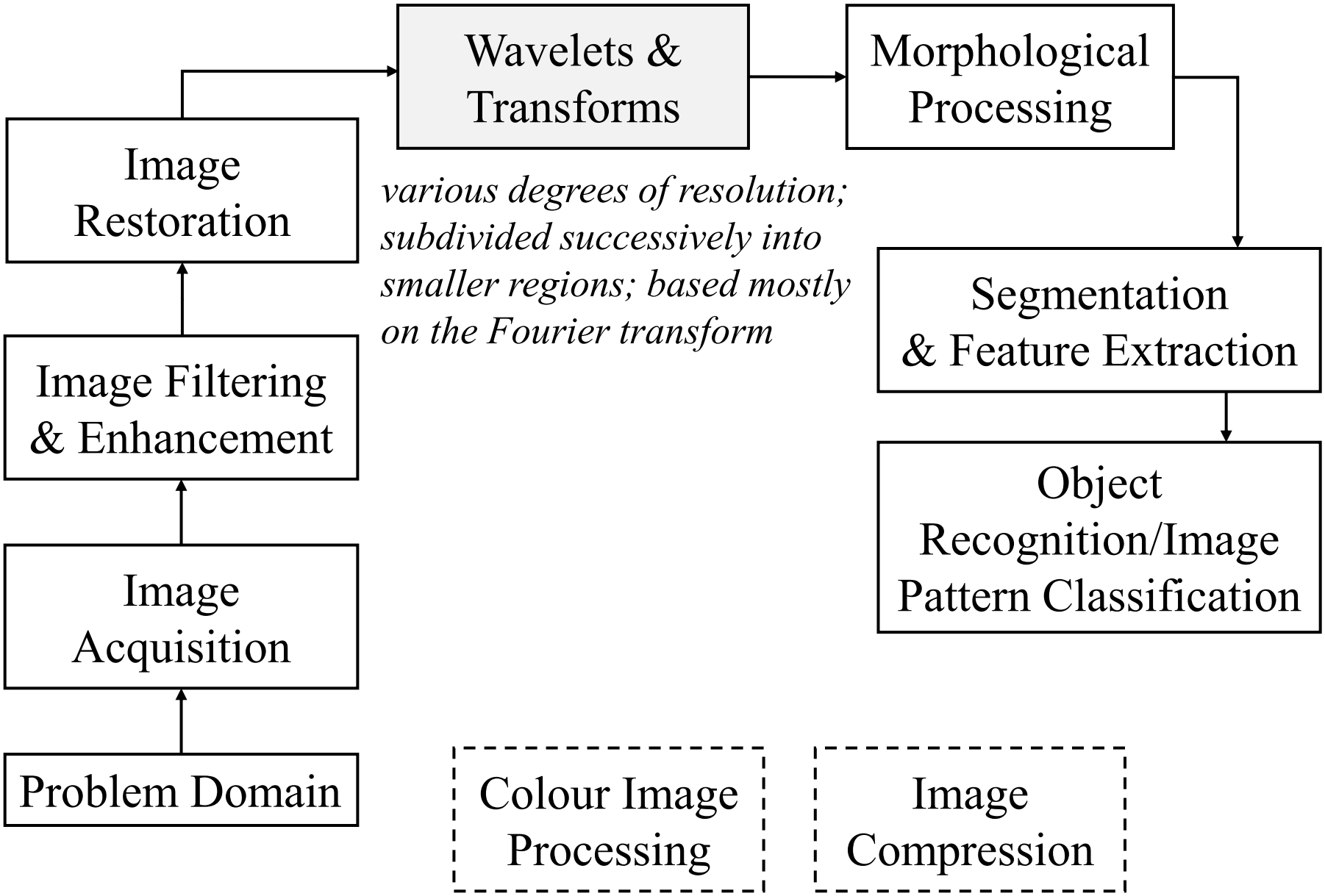
Colour Image Processing

Image Compression

Key Stages in DIP



Key Stages in DIP



various degrees of resolution; subdivided successively into smaller regions; based mostly on the Fourier transform

Key Stages in DIP

Problem Domain

Image Acquisition

Image Filtering & Enhancement

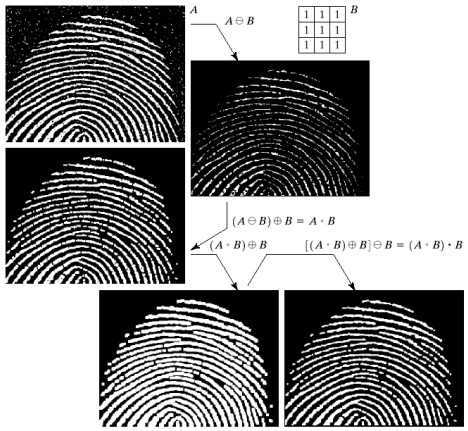
Image Restoration

Wavelets & Transforms

Morphological Processing

Segmentation & Feature Extraction

Object Recognition/Image Pattern Classification

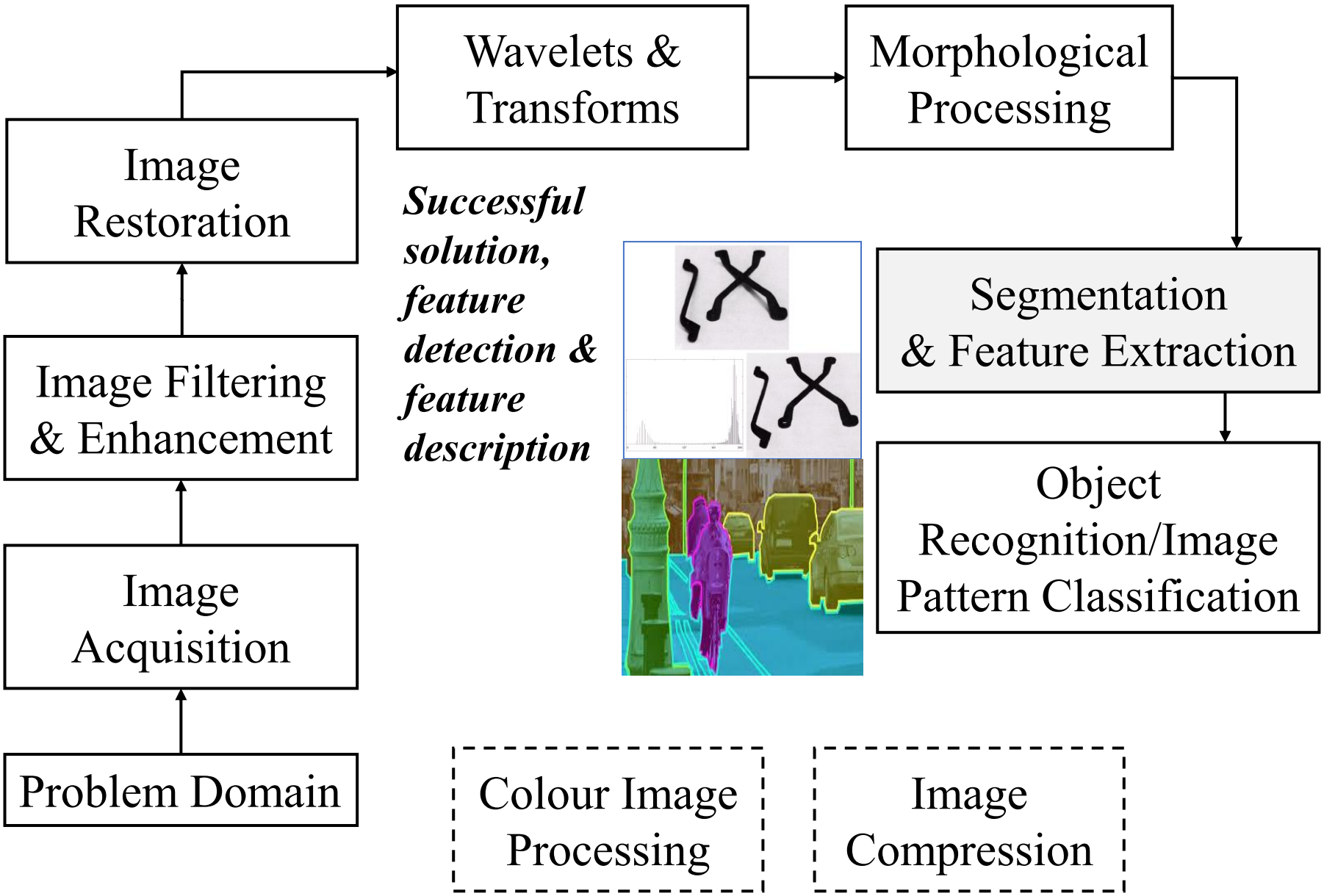


Representation & description of shape; Extracting image components

Colour Image Processing

Image Compression

Key Stages in DIP: Segmentation



Key Stages in DIP: **Object Recognition**

Problem Domain

Image Acquisition

Image Filtering & Enhancement

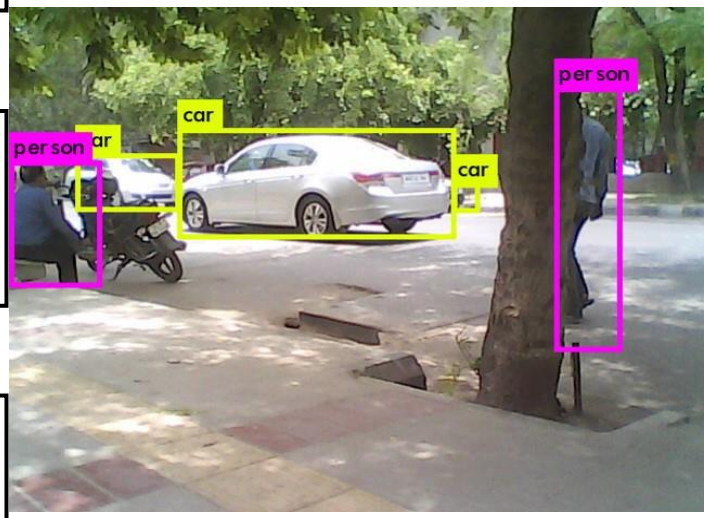
Image Restoration

Wavelets & Transforms

Morphological Processing

Segmentation & Feature Extraction

Object Recognition/Image Pattern Classification

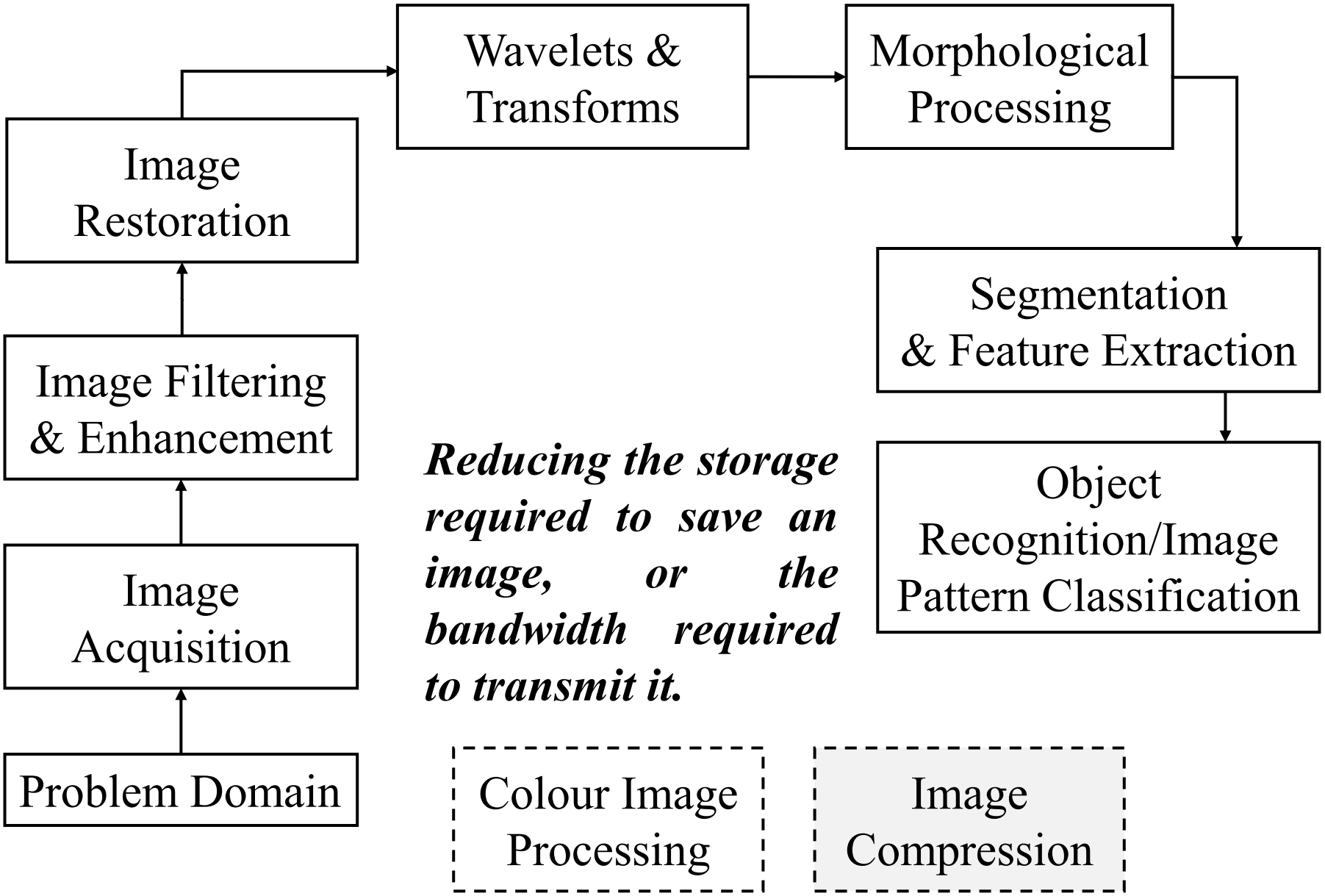


Minimum-distance, correlation, Bayes classifiers, deep neural networks, deep convolutional neural networks

Colour Image Processing

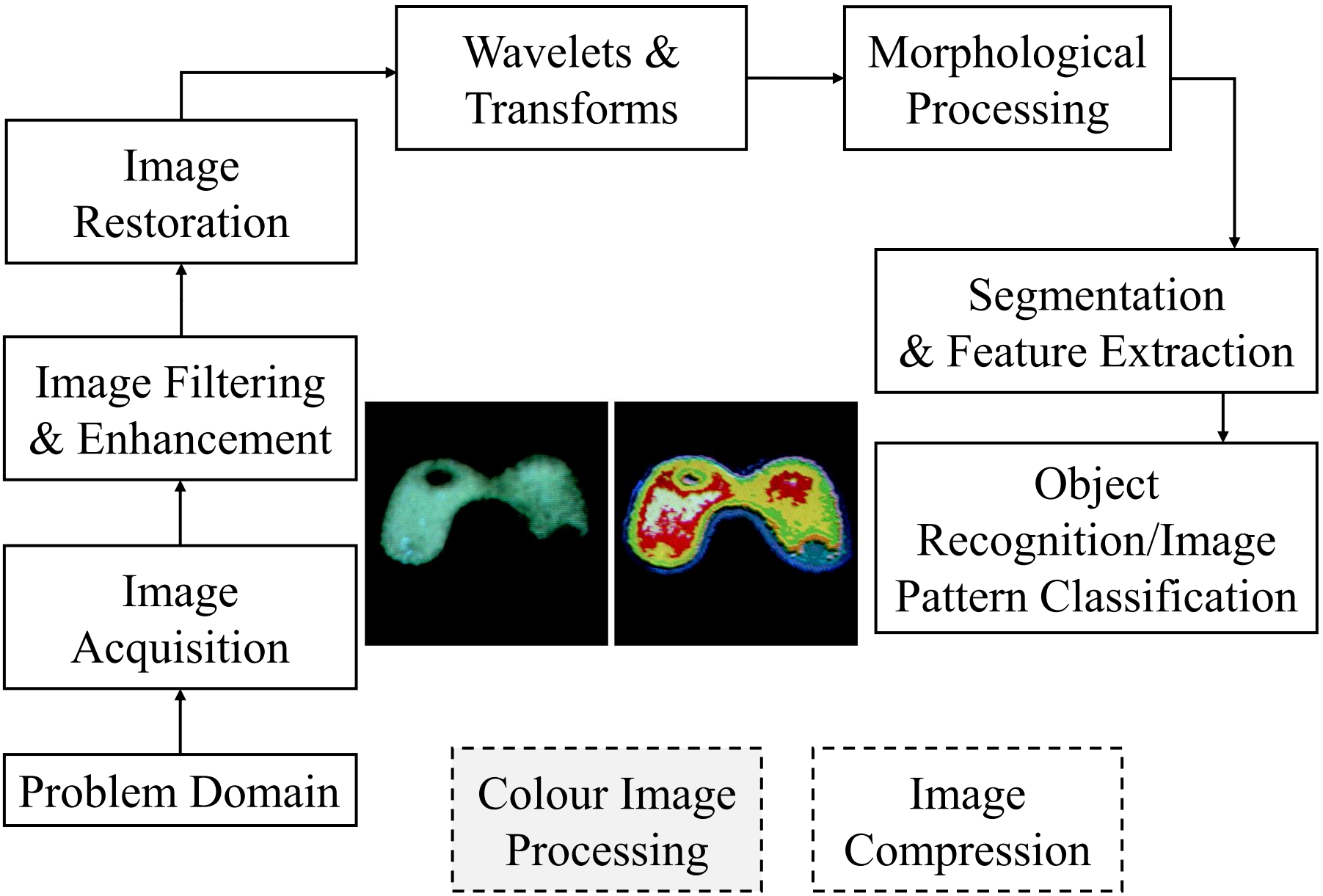
Image Compression

Key Stages in DIP: **Image Compression**



Reducing the storage required to save an image, or the bandwidth required to transmit it.

Key Stages in DIP: **Colour Image Processing**



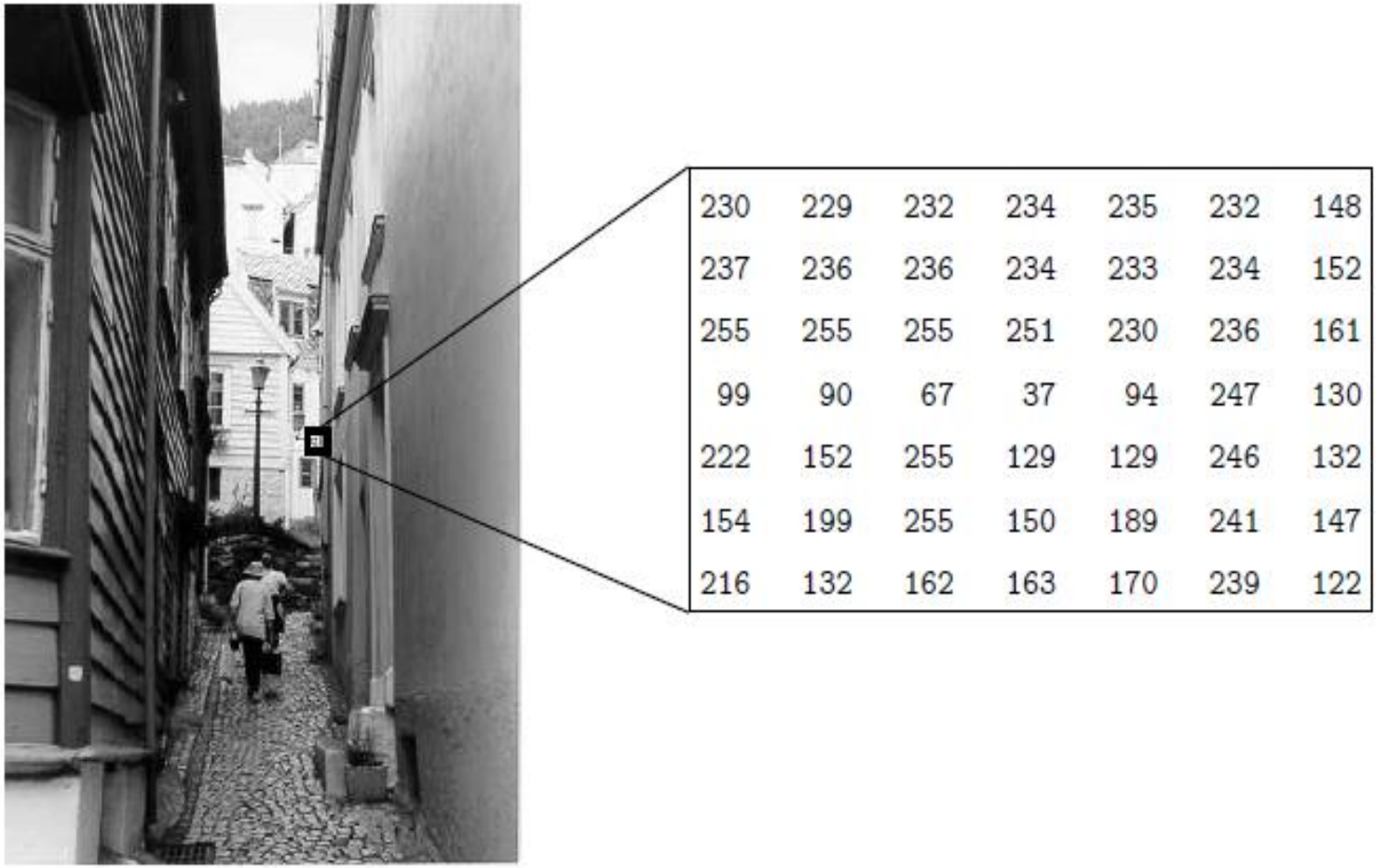
Types of Digital Images

1. Binary (0,1)
2. Greyscale (0 is Black to 255 is white)
3. True color, or RGB
4. Indexed

Types of Digital Images

1.Binary (0,1)

2.Greyscale (0 is Black to 255is white)



230	229	232	234	235	232	148
237	236	236	234	233	234	152
255	255	255	251	230	236	161
99	90	67	37	94	247	130
222	152	255	129	129	246	132
154	199	255	150	189	241	147
216	132	162	163	170	239	122

3.True color, or RGB



49	55	56	57	52	53
58	60	60	58	55	57
58	58	54	53	55	56
83	78	72	69	68	69
88	91	91	84	83	82
69	76	83	78	76	75
61	69	73	78	76	76

Red

64	76	82	79	78	78
93	93	91	91	86	86
88	82	88	90	88	89
125	119	113	108	111	110
137	136	132	128	126	120
105	108	114	114	118	113
96	103	112	108	111	107

Green

66	80	77	80	87	77
81	93	96	99	86	85
83	83	91	94	92	88
135	128	126	112	107	106
141	129	129	117	115	101
95	99	109	108	112	109
84	93	107	101	105	102

Blue

3. Indexed Image

Indexed color image value < RGB color value.

Image indices are images that are computed from multiband (RGB) images.



Indexed image uses direct mapping of pixel values to colormap values. The color of each image pixel is determined by using the corresponding value of X as an index into map .

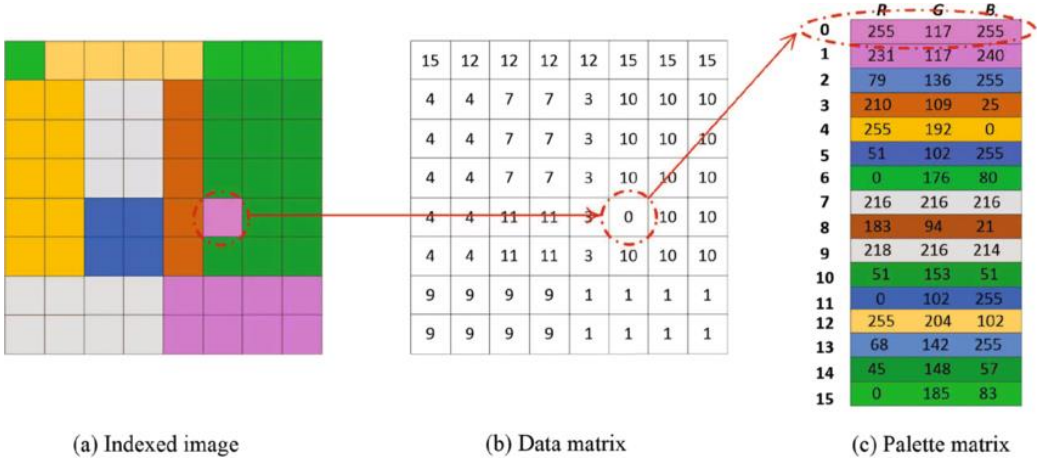
The value 1 points to the first row in map , the value 2 points to the second row, and so on.

3. Indexed Image

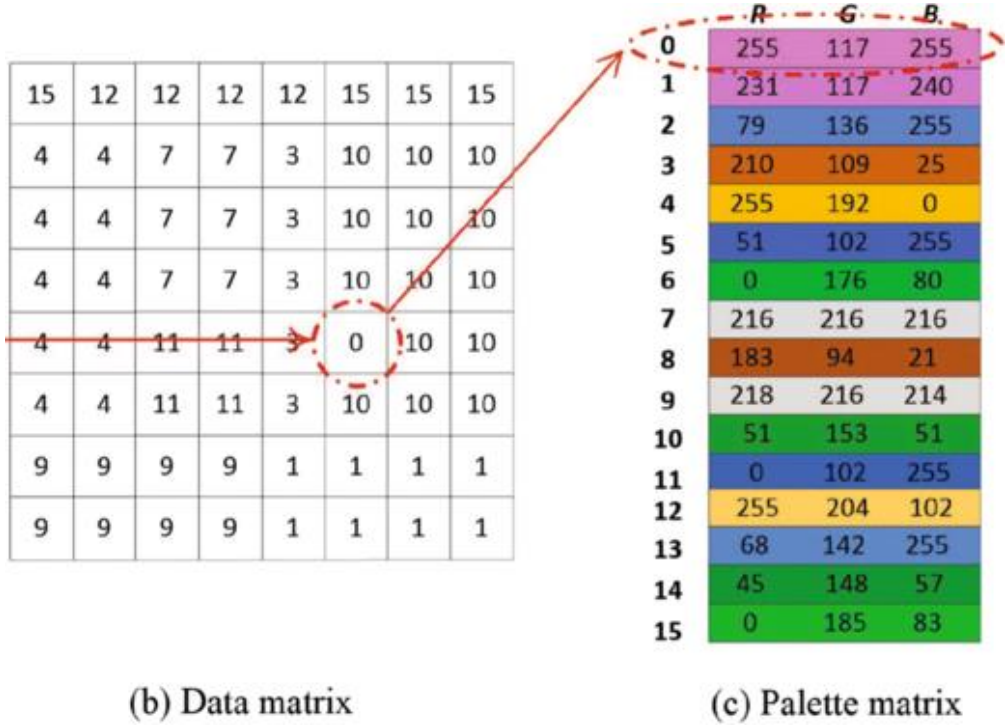
Indexed Image

It is a type of digital image where pixel values are **not stored as full color values (RGB)**, but instead as **indices** that reference a color in a **colormap (palette)**.

Instead of storing 24-bit RGB per pixel, we store an **index** (like 0,1,2,...) and then look up actual color in a palette.



(a) Indexed image (b) Data matrix (c) Palette matrix



(b) Data matrix (c) Palette matrix

Example of an Indexed Image

Suppose we have a **2×3 image**: Pixel index matrix, $I = \begin{bmatrix} 0 & 1 & 2 \\ 2 & 1 & 0 \end{bmatrix}$

- Colormap (RGB):

Pixel Index	Colormap (palette)	Color
0	(255,0,0)	Red
1	(0,255,0)	Green
2	(0,0,255)	Blue

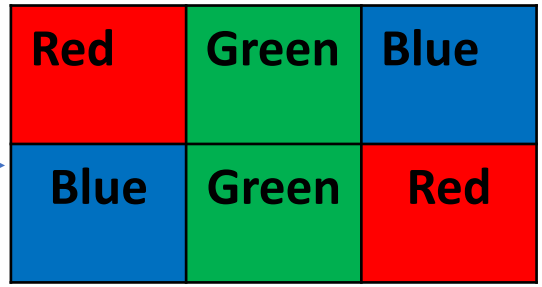
Pixel Index	Color
0	Red
1	Green
2	Blue

Indexed image = pixel indices + colormap.
Instead of storing full color for each pixel, you store a small number (index) and look it up in a palette.

0	1	2
2	1	0

Color/Data matrix

$$I = \begin{bmatrix} Red & Green & Blue \\ Blue & Green & Red \end{bmatrix}$$



Indexed Image

Why Indexed Images?

- ✓ Memory efficient (only store indices, not full RGB values).
- ✓ Useful for GIFs, icons, older graphics systems.
- ✓ Easier compression when colors are limited.

- ✓ **Limitation**: Color range is limited to the colormap (commonly 256 colors).

Thank You

Lecture 1.3

Course Name: Digital Image Processing
Course code: 4245

Digital Image Fundamentals

Instructor Name:
Professor Dr. Jahangir Alam
Professor, Dept. of CSE, BAUST

Representation of Digital Image

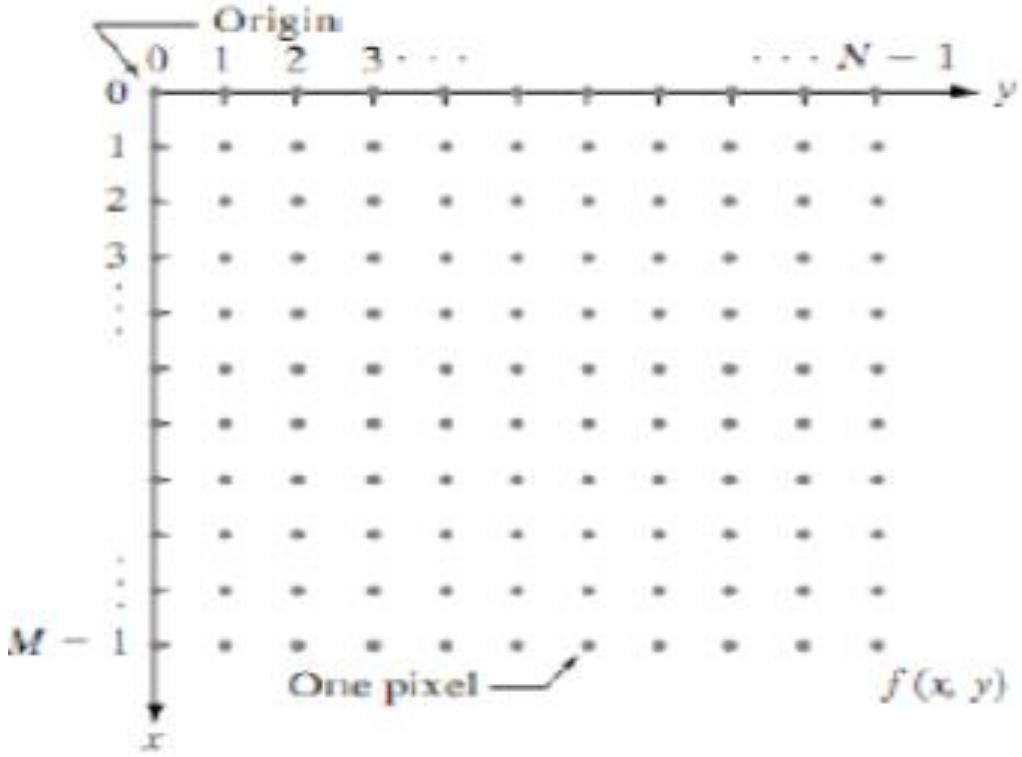
- An image 2D function as, $f(x, y)$

  where x and y are *spatial* (plane) coordinates, and the amplitude of f at any pair of coordinates (x, y).

- f is called the *intensity* or *gray level* of the image at that point.
- When x, y, and the intensity values of f are all finite, discrete quantities, we call the image a *digital image*.
- The field of *digital image processing* refers to processing digital images by means of a digital computer.

Representation of Digital Image

$$f(x,y)=\begin{bmatrix} f(0,0) & f(0,1) & \cdots & f(0,N-1) \\ f(1,0) & f(1,1) & \cdots & f(1,N-1) \\ \vdots & \vdots & & \vdots \\ f(M-1,0) & f(M-1,1) & \cdots & f(M-1,N-1) \end{bmatrix}$$



Representing Digital Images

$M \times N$ *numerical arrays*:

$$f(x,y) = \begin{bmatrix} f(0,0) & f(0,1) & \cdots & f(0,N-1) \\ f(1,0) & f(1,1) & \cdots & f(1,N-1) \\ \vdots & \vdots & & \vdots \\ f(M-1,0) & f(M-1,1) & \cdots & f(M-1,N-1) \end{bmatrix}$$

$$a_{ij} = f(i,j)$$

pixel with coordinates

$$\mathbf{A} = \begin{bmatrix} a_{0,0} & a_{0,1} & \cdots & a_{0,N-1} \\ a_{1,0} & a_{1,1} & \cdots & a_{1,N-1} \\ \vdots & \vdots & & \vdots \\ a_{M-1,0} & a_{M-1,1} & \cdots & a_{M-1,N-1} \end{bmatrix}$$

Representing Digital Images

$L = 2^k$ ← Integer

$[0, L-1]$

(lower limit) *noise*

(upper limit) *saturation*

Image contrast

>> help contrast

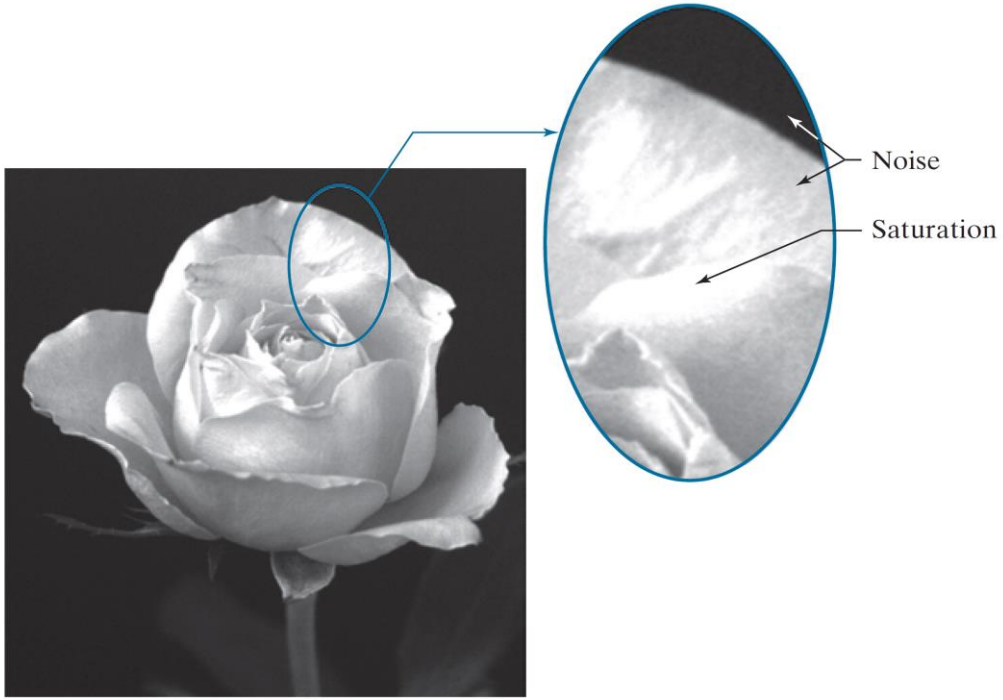


Image Model and Image Feature Vector

Extract simple features manually,

- Sum of white pixels in each row
- Sum of white pixels in each column

$$I = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 255 & 0 \\ 0 & 255 & 0 \end{bmatrix}$$

Row sums:

. Row 1: 0 + 0 + 0 = 0

. Row 2: 0 + 255 + 0 = 255

. Row 3: 0 + 255 + 0 = 255

Vector: [0, 255, 255]

Column sums:

. Col 1: 0 + 0 + 0 = 0

. Col 2: 0 + 255 + 255 = 510

. Col 3: 0 + 0 + 0 = 0

Vector: [0, 510, 0]

Concatenate row and column sums into a feature vector:

F=[0, 255, 255, 0, 510, 0]

Image Classification

Assume we have two classes: digit "**1**" or "**0**".

Rule example:

- If column 2 sum is high (> 400) and row 2 and 3 sums are high (>200), classify as "1".
- Else classify as "0".

$$I = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 255 & 0 \\ 0 & 255 & 0 \end{bmatrix}$$

- Column 2 sum = 510 > 400
- Row 2 sum = 255 > 200
- Row 3 sum = 255 > 200

So, classify image as digit "1".

Image Classification

Assume we have two classes: digit "1" or "0".

Rule example:

- If column 2 sum is high (> 400) and row 2 and 3 sums are high (>200), classify as "1".
- Else classify as "0".

- Column 2 sum = 255 < 400
- Row 2 sum= 510> 200
- Row 3 sum = 255 > 200

$$I_0 = \begin{bmatrix} 255 & 0 & 255 \\ 255 & 0 & 255 \\ 0 & 255 & 0 \end{bmatrix}$$

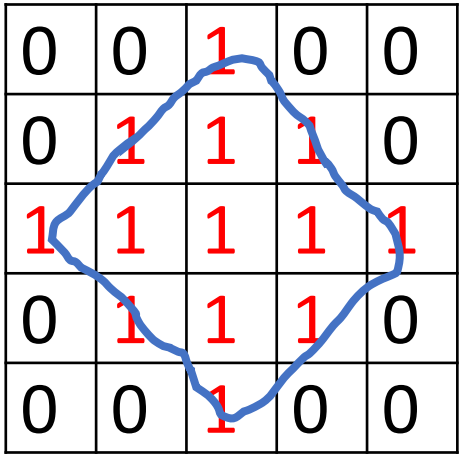
So, classify image as digit “0".

Image Representation and Description

Pixel-based representation:

List coordinates of foreground pixels (value = 1):

{(0,2), (1,1), (1,2), (1,3), (2,0), (2,1), (2,2), (2,3), (2,4), (3,1), (3,2), (3,3), (4,2)}



Boundary representation:

List boundary pixels (outer edge of shape):

{(0,2), (1,1), (1,3), (2,0), (2,4), (3,1), (3,3), (4,2)}

Description (Basic shape features)

1. Area: Count of foreground pixels = 13

2. Perimeter: Count of boundary pixels = 8

Image Representation and Description

Pixel-based representation:

List coordinates of foreground pixels (value = 1):

$\{(0,2), (1,1), (1,2), (1,3), (2,0), (2,1), (2,2), (2,3), (2,4), (3,1), (3,2), (3,3), (4,2)\}$

0	0	1	0	0
0	1	1	1	0
1	1	1	1	1
0	1	1	1	0
0	0	1	0	0

Description (Basic shape features)

Thus, centroid ≈ (1.85, 2.0)

1. Centroid:

$$x_c = \frac{0+1+1+1+2+2+2+2+2+3+3+3+4}{13} \approx 1.85$$

$$y_c = \frac{2+1+2+3+0+1+2+3+4+1+2+3+2}{13} = 2.0$$

Image Representation and Description

{(0,2), (1,1), (1,2), (1,3), (2,0), (2,1), (2,2), (2,3), (2,4), (3,1), (3,2), (3,3), (4,2)}

0	0	1	0	0
0	1	1	1	0
1	1	1	1	1
0	1	1	1	0
0	0	1	0	0

Bounding box:

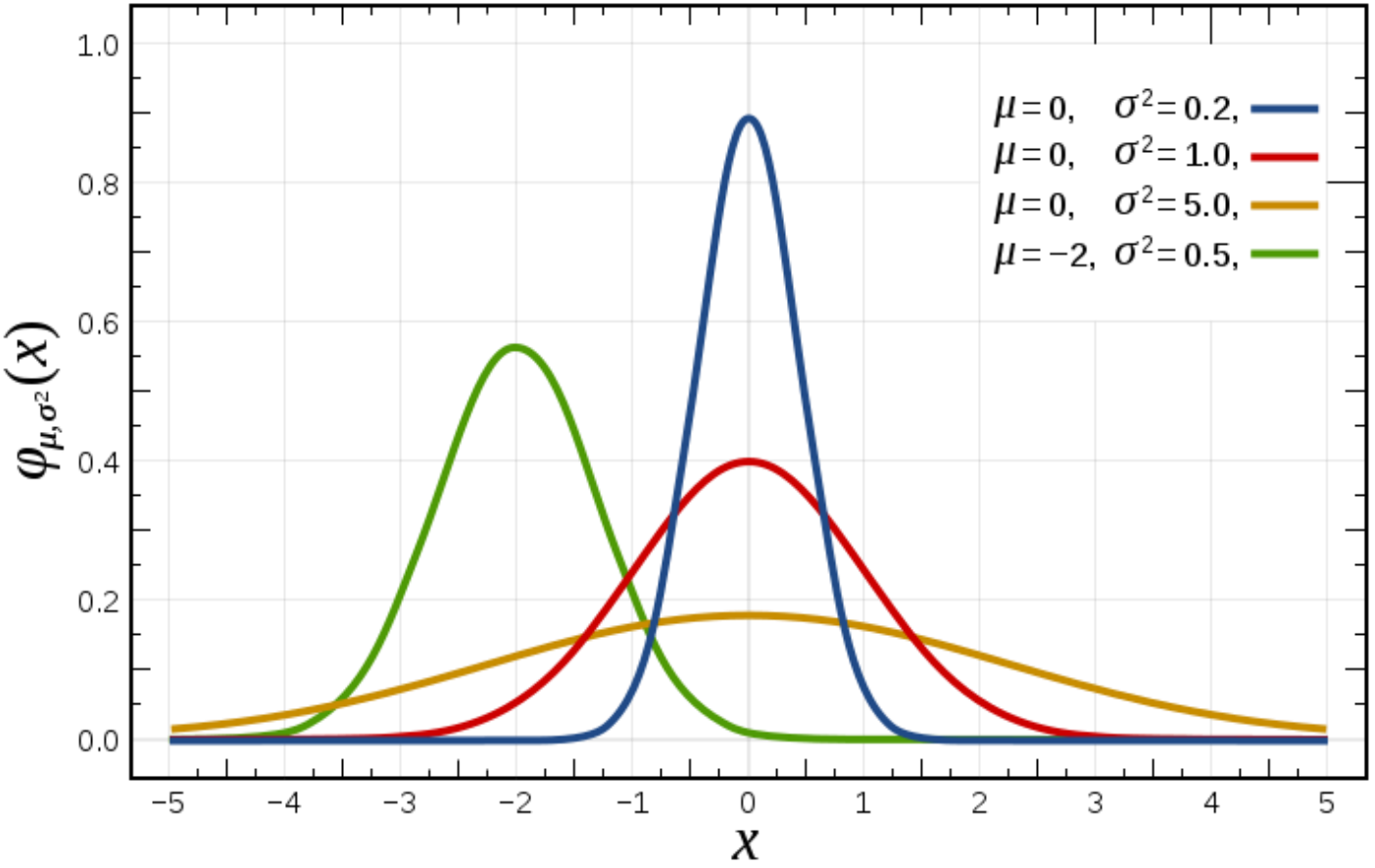
- Xmin = 0, Xmax = 4
- Ymin = 0, Ymax = 4

Summary:

Descriptor	Value
Area	13 pixels
Perimeter	8 pixels
Centroid	(1.85, 2.0)
Bounding box	From (0,0) to (4,4)

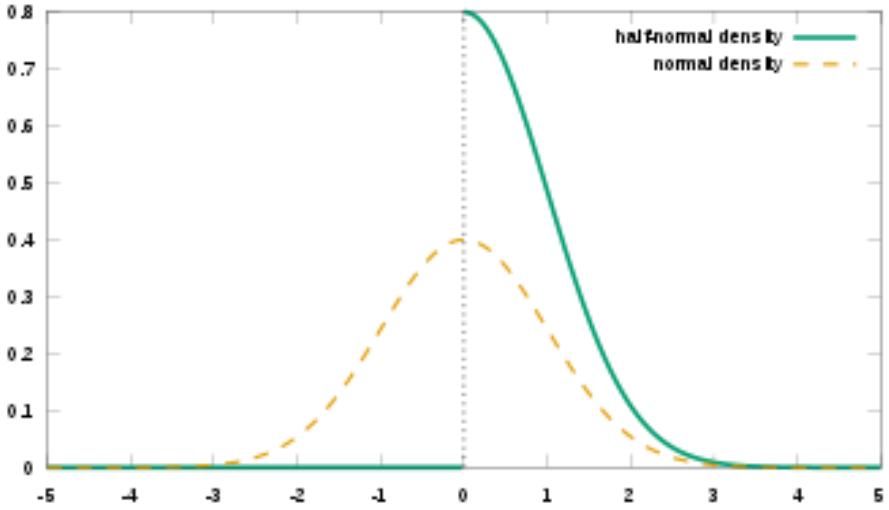
Normal or Gaussian distribution

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

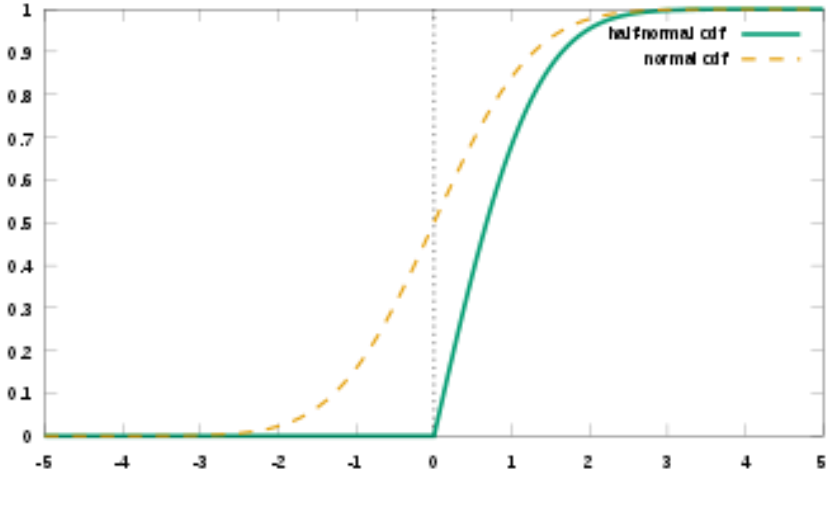


Half-normal Distribution

PDF	$f(x;\sigma) = \frac{\sqrt{2}}{\sigma\sqrt{\pi}} \exp\left(-\frac{x^2}{2\sigma^2}\right) \quad x > 0$
CDF	$F(x;\sigma) = \operatorname{erf}\left(\frac{x}{\sigma\sqrt{2}}\right)$



PDF



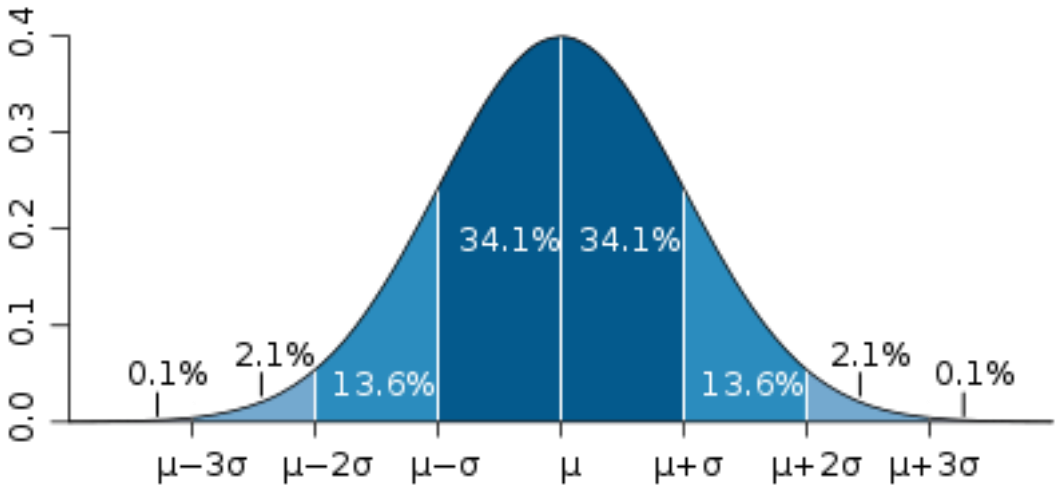
CDF

Standard Normal Distribution

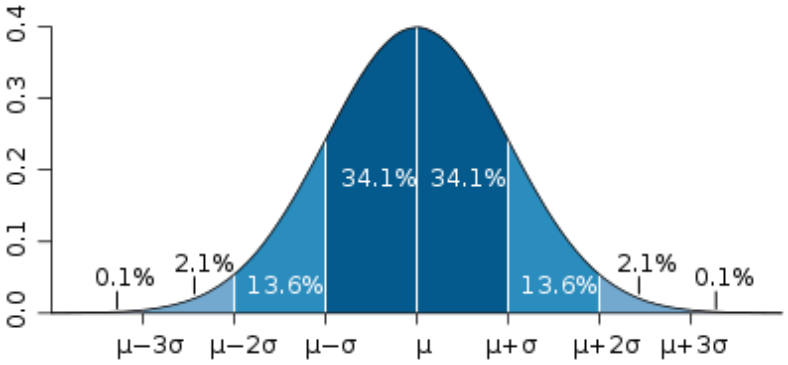
$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

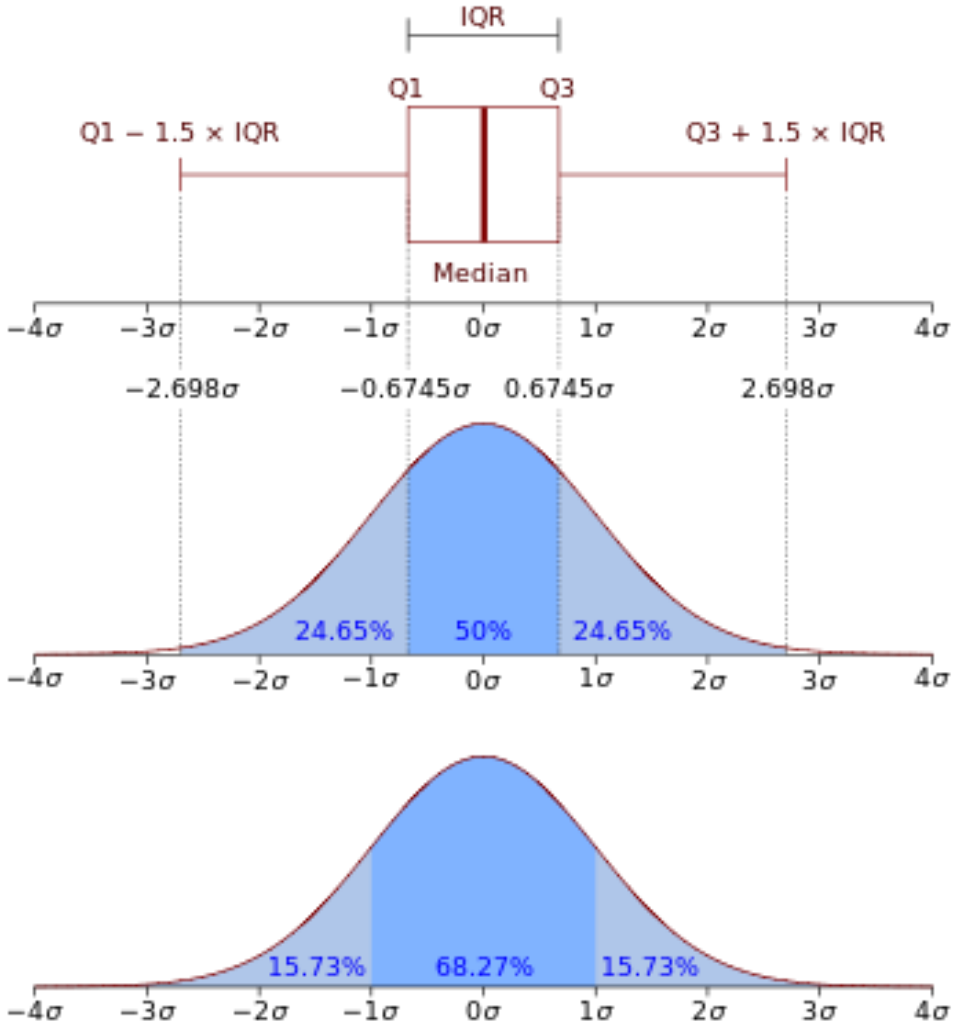
$$\mu = 0, \sigma = 1$$

$$\varphi(z) = \frac{e^{-z^2/2}}{\sqrt{2\pi}}.$$



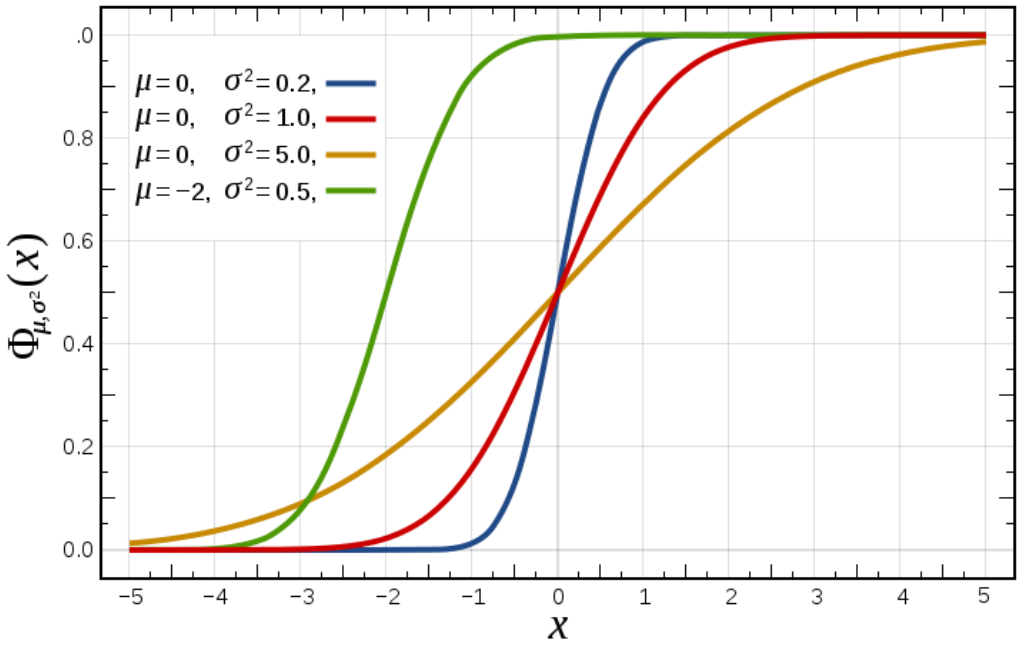
PDF of Normal Distribution





CDF of Normal Distribution

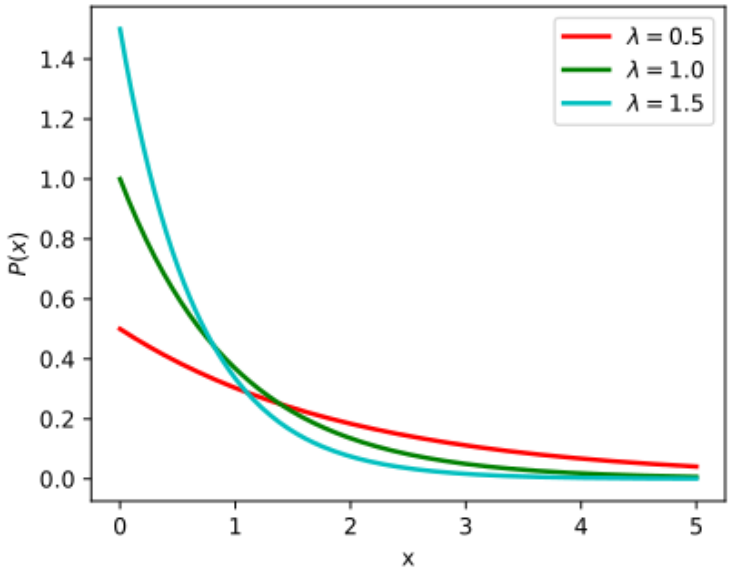
$$\Phi(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{x} e^{-t^2/2}\, dt$$

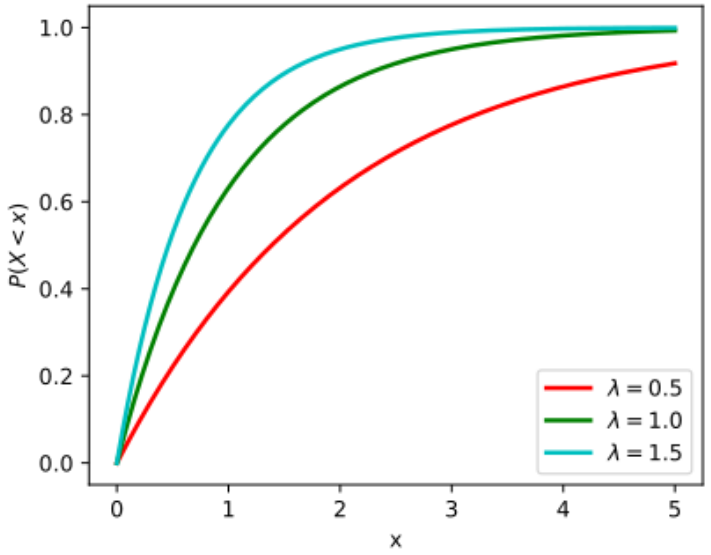


Suppose X is normal distributed. Then the CDF of X is given by

$$F(x;\mu,\sigma) = \frac{1}{\sigma\sqrt{2\pi}} \int_{-\infty}^{x} \exp\left(-\frac{(t-\mu)^2}{2\sigma^2}\right) dt.$$

PDF & CDF of Exponential Distribution





$$f(x;\lambda) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0, \\ 0 & x < 0. \end{cases}$$

$$F(x;\lambda) = \begin{cases} 1 - e^{-\lambda x} & x \geq 0, \\ 0 & x < 0. \end{cases}$$

$\lambda > 0$ is the parameter of the distribution, often called the *rate parameter*.

Gaussian Noise in Image

Gaussian noise values N (mean = 0, std dev = 2)

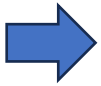
Assume, noise values (drawn from $\mathcal{N}(0, 2^2)$):

1	-2	0
0	2	-1
-2	1	1

100	102	98
101	99	103
97	100	102

Noisy Image:

100+1	102-2	98+0
101+0	99+2	103-1
97-2	100+1	102+1



101	100	98
101	101	102
95	101	103

- Original pixel values were close to 100.
- Added Gaussian noise with small variance.
- Resulting image values are slightly perturbed, simulating noise.

Parametric Method in Image Processing

- Foreground sample pixels: [180, 185, 190, 175, 195]
- Background sample pixels: [40,45,50,35,55]

Foreground:

$$\mu_F = \frac{180 + 185 + 190 + 175 + 195}{5} = 185$$

$$\sigma_F^2 = \frac{\sum(x_i - \mu_F)^2}{n - 1} = \frac{250}{4} = 62.5$$

Background:

$$\mu_B = \frac{40 + 45 + 50 + 35 + 55}{5} = 45$$

$$\sigma_B^2 = \frac{\sum(x_i - \mu_B)^2}{n - 1} = \frac{250}{4} = 62.5$$

Parametric Method in Image Processing

- Foreground sample pixels: [180, 185, 190, 175, 195]
- Background sample pixels: [40,45,50,35,55]

Classify a New Pixel Intensity $x = 170$

$$p_F = \frac{1}{\sqrt{2\pi \times 62.5}} e^{-\frac{(170-185)^2}{2\times 62.5}} \approx 0.0083$$

$$p_B = \frac{1}{\sqrt{2\pi \times 62.5}} e^{-\frac{(170-45)^2}{2\times 62.5}} \approx 0$$

Since, $pF > pB$, classify pixel with intensity 170 as **foreground**.

Entropy, Image Quality

- Entropy measures the randomness or information content in an image.
- Higher entropy = more details and textures (more complex image).
- Lower entropy = less detail (more uniform or flat image).

$$\begin{bmatrix} 0 & 0 & 255 \\ 0 & 255 & 255 \\ 255 & 255 & 255 \end{bmatrix}$$

Pixel Value	Count	Probability
0	3	3/9=0.333
255	6	6/9=0.666

Entropy, Image Quality

Pixel Value	Count	Probability
0	3	3/9=0.333
255	6	6/9=0.666

Total Entropy

$$H = -\sum_i P(i)\log_2 P(i)$$

$$= -P(0)\times\log_2 P(0) - P(255)\times\log_2 P(255)$$

$$= -0.333\times\log_2(0.333) - 0.666\times\log_2(0.666)$$

$$= 0.528 + 0.389 = 0.917\ bits$$

Image Type	Entropy (bits)	Quality Interpretation
Uniform (all same)	0	No detail, low quality
Mixed (like above)	~0.9	Good variation, decent quality
Max. variation	1 (for 2 levels)	Highest detail/texture possible

Chebyshev, Euclidean, Manhattan, Minkowski Distance

Distance	Formula	Applicability
Chebyshev Distance	$D_{Chebyhev}(p,q) = max(\lvert p_1 - q_1\rvert, \lvert p_2 - q_2\rvert, \dots, \lvert p_n - q_n\rvert)$	Useful for grid-based systems with free movement, like chess (king's moves) and certain robotic paths.
Euclidean Distance	$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$	Ideal for continuous spaces where straight-line distance is required, like geometry and physics.
Manhattan Distance	$\lvert x_1 - x_2\rvert + \lvert y_1 - y_2\rvert$	Relevant in grids with only horizontal and vertical movement, such as city blocks or chess (Rook's moves).
Minkowski distance	$\sqrt[p]{\lvert x_1 - y_1\rvert^p + \lvert x_2 - y_2\rvert^p + \lvert x_2 -}$	A flexible metric for various multidimensional spaces; often used in machine learning and clustering, for various values of p.

Distance Measures

##Let, p and q are two pixels as shown in Table 5b. Calculate city block distance (D4), chessboard distance (D8), and Euclidean distance (De). The set V is given V={2,4}. Compute length of the shortest path-4, shortest path-8 and shortest path-m between pixels p and q, where V={2,4}.

	0	1	2	3
0	4	3	2	2
1	4	3	2	1
2	1	3	2	1
3	2	4	1	0

q(s,t)=q(0,3)

P(x,y)=P(3,0)

Distance	Formula
Chebyshev Distance (Chess Board Distance)	$D_{Chebyhev}(p,q) = max(\|0-3\|, \|3-0\|)$ $= max(3,3) = 3$
Euclidean Distance	$D_e = \sqrt{(0-3)^2 + (3-0)^2} = \sqrt{18} = 3\sqrt{2}$
Manhattan Distance (City Block Distance)	$D_{cb} = \|0-3\| + \|3-0\| = 6$
Minkowski distance	$D_m = \sqrt[3]{\|0-3\|^3 + \|3-0\|^3} = \sqrt[3]{54} = 3\sqrt{2}$

Distance Measures

City Block Distance: The D_4 distance between p & q is defined as

$$D_4(p, q) = |x - s| + |y - t|$$

In this case, pixels having D_4 distance from (x, y) less than or equal to some value r form a diamond centered at (x, y).

```
        2
    2   1   2
2   1   0   1   2
    2   1   2
        2
```

The *City block distance* is always greater than or equal to zero.

Pixels with D_4 distance ≤ 2 forms the following contour of constant distance.

Distance Measures

Chess-Board Distance: The D_8 distance between p & q is defined as

$$D_8(p, q) = \max(|x - s|, |y - t|)$$

In this case, pixels having D_8 distance from (x, y) less than or equal to some value r form a square centered at (x, y).

2	2	2	2	2
2	1	1	1	2
2	1	0	1	2
2	1	1	1	2
2	2	2	2	2

Pixels with D_8 distance ≤ 2 forms the following contour of constant distance.

Thank You

Lecture 1.4

Course Name: Digital Image Processing
Course code: 4245

Relationships between Pixels & Image Transforms

Instructor Name:
Professor Dr. Jahangir Alam
Professor & Head,
Dept. of CSE, BAUST

Regions & Boundaries of an Image

$$I = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \end{bmatrix}$$

- **Region 1:** Pixels: (2,2),(2,3),(3,2),(3,3)

Check neighbors: All 4 pixels touch background (0) at some side → **all are boundary pixels**.

- **Region 2:** Pixels: (4,4),(4,5),(5,4),(5,5)
- All pixels are adjacent to background → **all boundary pixels.**

Digital Depth (or Curve) Image

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

- **1** = object pixel
- **0** = background pixel

Identify the Boundary Pixels (Depth = 1)

Boundary pixels are object pixels adjacent (8-neighbors) to at least one background pixel (0).

Mark boundary pixels with depth 1:

Explanation:

- All outer '1's are next to 0, so depth = 1.
- The center pixel at (3,3) is not adjacent to 0, so it remains 0 for now.

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Digital Depth (or Curve) Image

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 2 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Identify Pixels with Depth = 2

Pixels fully surrounded by depth 1 pixels and inside object get depth = 2.
Center pixel (3,3) neighbors:

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & ? & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Since all neighbors of center pixel are depth 1 pixels, assign:

$$D(3,3) = 2$$

Digital Depth Map

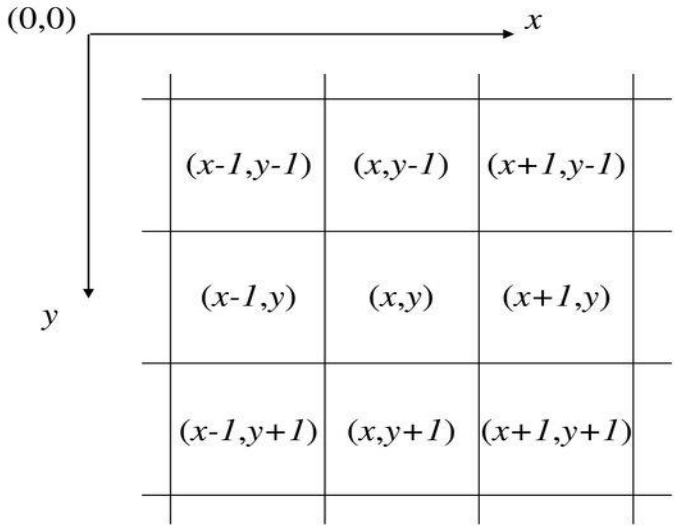
$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 2 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Digital Depth (or Curve)

Sequence of distinct pixels from $p(x_0, y_0)$ to $q(x_n, y_n)$

$$(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n)$$

Basic Relationship of Pixels

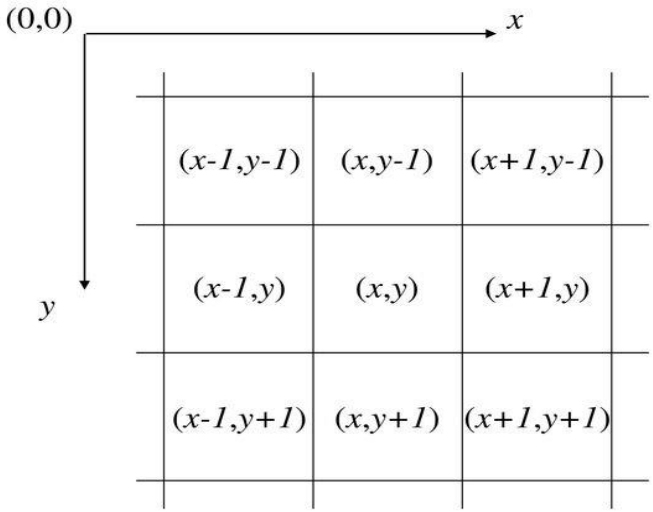


Conventional indexing method

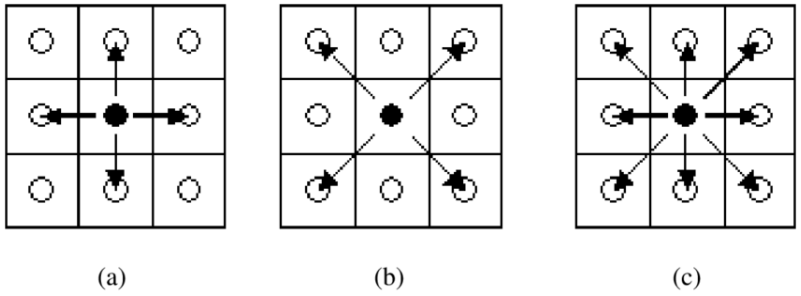
Digital Depth (or Curve)

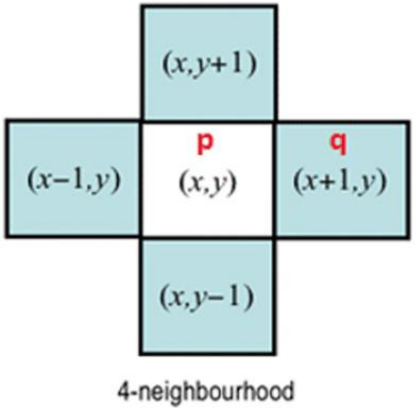
Basic Relationship of Pixels

$4-path, 8-path, \ m-path$

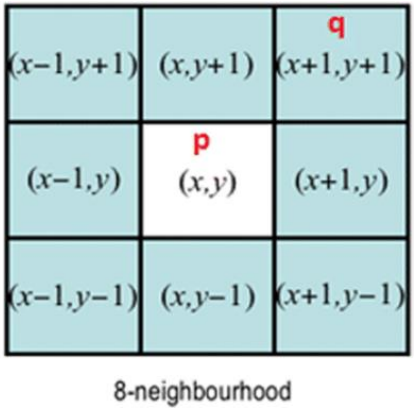


Conventional indexing method





4-neighbourhood



8-neighbourhood

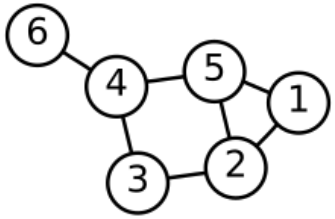
Neighbourhood (graph theory)

- In graph theory, an **adjacent vertex** of a vertex *v* in a graph is a vertex that is connected to *v* by an edge.

- The **neighbourhood** of a vertex *v* in a graph *G* is the subgraph of *G* induced by all vertices adjacent to *v*, i.e., the graph composed of the vertices adjacent to *v* and all edges connecting vertices adjacent to *v*.

- In this graph, the vertices adjacent to 5 are 1, 2 and 4.

- The neighbourhood of 5 is the graph consisting of the vertices 1, 2, 4 and the edge connecting 1 and 2.



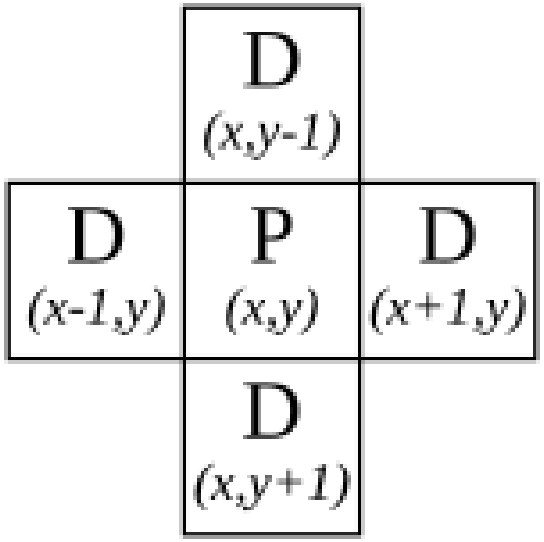
Adjacency, Connectivity

Intensity for binary image: $V = \{0 - 1\}$

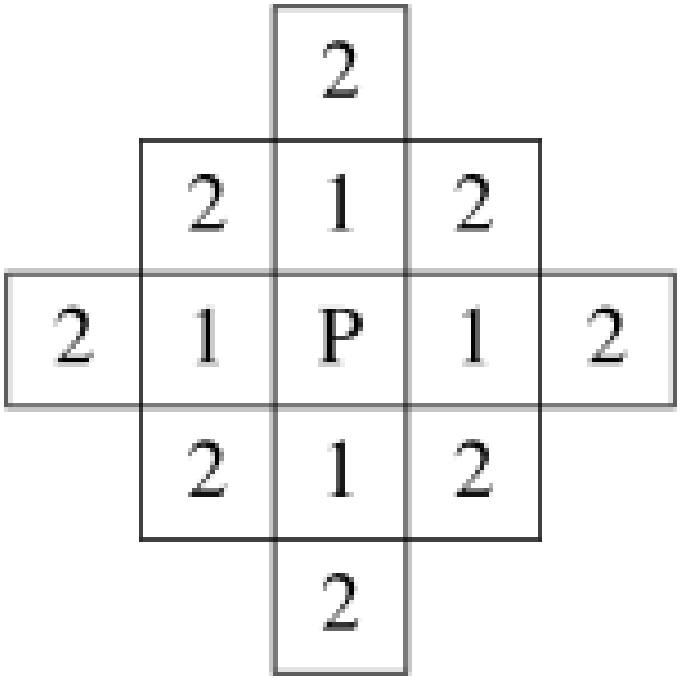
Intensity for grayscale image: $V = \{0 - 255\}$

1. 4-*adjacency*. Two pixels p and q with values from V are 4-adjacent if q is in the set $N_4(p)$.
2. 8-*adjacency*. Two pixels p and q with values from V are 8-adjacent if q is in the set $N_8(p)$.
3. *m-adjacency* (also called *mixed adjacency*). Two pixels p and q with values from V are m-adjacent if
   (a) q is in $N_4(p)$, *or*
   (b) q is in $N_D(p)$ *and* the set $N_4(p) \cap N_4(q)$ has no pixels whose values are from V.

Von Neumann neighborhood (or 4-neighborhood)



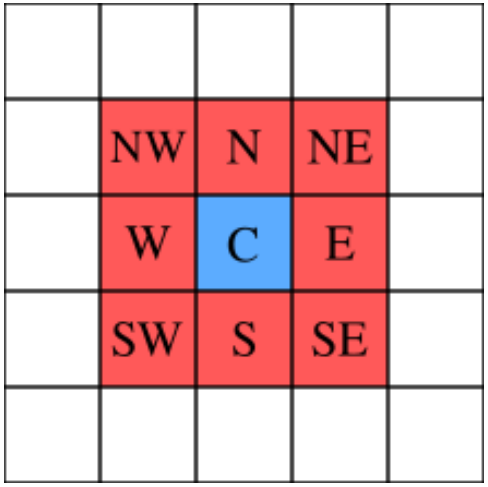
Manhattan distance r = 1



Manhattan distance r = 2

Moore neighborhood (8-connected pixels)

The Moore neighborhood is composed of nine cells: a central cell and the eight cells which surround it.



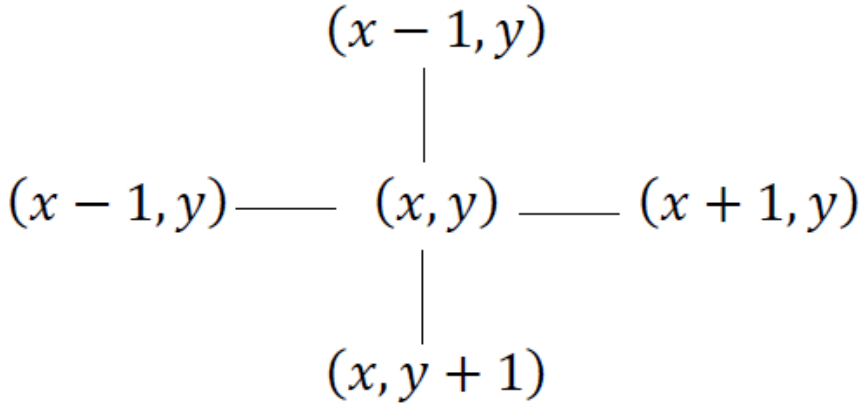
In two dimensions, the number of cells in an *extended* Moore neighbourhood of range r is $(2r + 1)^2$ while the number of cells with Chebyshev distance r is $(2r+1)^2-(2r-1)^2$.

$$d_{\text{Chebyshev}}(P, Q) = \max_i(|p_i - q_i|)$$

Conway's Game of Life?

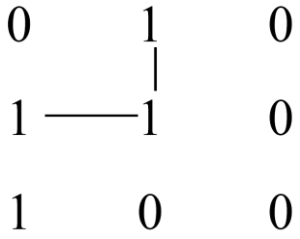
4-Neighbors of Pixel

A pixel p at coordinates (x, y) has two horizontal and two vertical neighbors with coordinates.

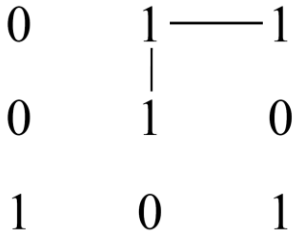


4-connectivity: Two or more pixels are said to be 4-connected if they are 4-adjacent with each other's. $N4$(p): Called 4-*neighbors* of *p*.

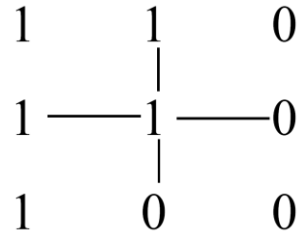
4-adjacency: Two pixels p and with the values from set 'V' are 4-adjacent if q is in the set of N4(p).



$V = \{1\}$



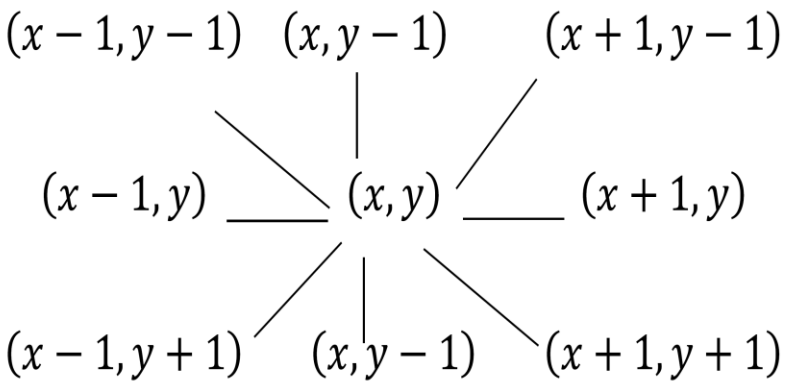
$V = \{1\}$



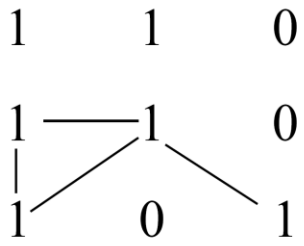
$V = \{0,1\}$

8-Neighbors of Pixel

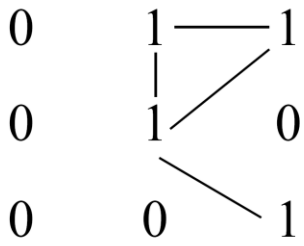
Two or more pixels are said to be 8-connected if they are 8-adjacent with each other. The four *diagonal* neighbors of p have coordinates. *4 diagonal neighbors of p +4-neighbors of p. N8(p): Called 8-neighbors of p.*



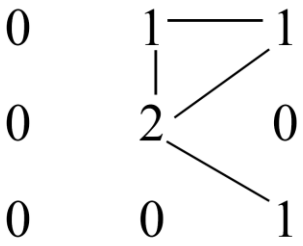
8-adjacency: Two pixels p and with the values from set 'V' are 8-adjacent if q is in the set of N8(p)



$V = \{1\}$



$V = \{1\}$

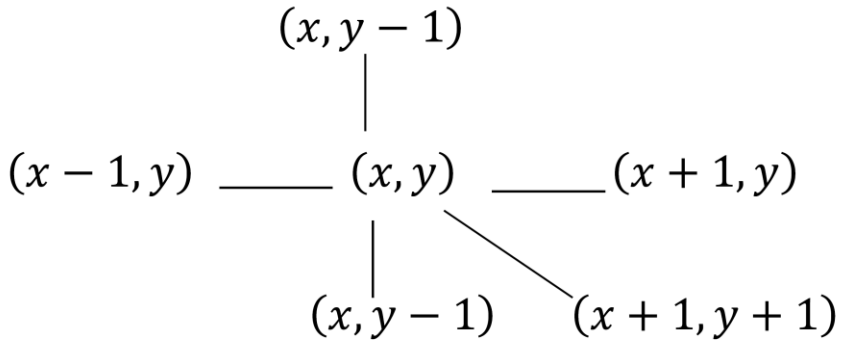


$V = \{1,2\}$

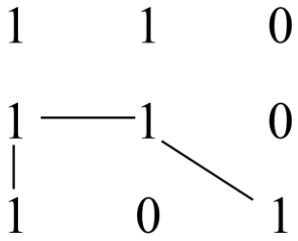
m-Neighbors of Pixel

Two or more pixels are said to be m-connected if they are m-adjacent with each others.

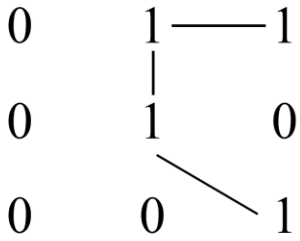
*N*m(p): Called m-*neighbors* of *p*.



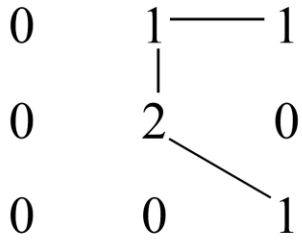
m-adjacency: Two pixels p and with the values from set 'V' are m-adjacent if q is in the set of Nm(p)



$V = \{1\}$



$V = \{1\}$



$V = \{1,2\}$

Paths

##Consider the image segment shown in the following. Compute the length of the Shortest-4, Shortest-8, and Shortest-m paths between pixels p and q. where, $V = \{1, 2\}$.

4	2	3	2 (q)
3	3	1	3
2	3	2	2
(p)2	1	2	3

Solution:

Shortest-4 path, V={1,2}: Path does not exist

4	2	3	**2 (q)**
3	3	1	3
2	3	2	2
(p)2	1	2	3

$V = \{1,2\}$

4	2	3	**2 (q)**
3	3	1	3
2	3	2	2
(p)2→	1	2	3

$V = \{1,2\}$

4	2	3	**2 (q)**
3	3	1	3
2	3	2	2
(p)2→	1	→ 2	3

$V = \{1,2\}$

Paths

Shortest-8 path, V={1,2}: Path exists ⇒ shortest path 4

4	2	3	**2 (q)**
3	3	1	3
2	3	2	2
(p)2	1	2	3

$V = \{1,2\}$

4	2	3	**2 (q)**
3	3	1	3
2	3	2	2
(p)2→	1	2	3

$V = \{1,2\}$

4	2	3	**2 (q)**
3	3	1	3
2	3	2	2
(p)2→	1	2	3

$V = \{1,2\}$

Shortest-m path, V={1,2}: Path exists ⇒ shortest path 5

4	2	3	**2 (q)**
3	3	1	3
2	3	2	2
(p)2	1	2	3

$V = \{1,2\}$

4	2	3	**2 (q)**
3	3	1	3
2	3	2	2
(p)2→	1	2	3

$V = \{1,2\}$

4	2	3	**2 (q)**
3	3	1	3
2	3	2	2
(p)2→	1	→ 2	3

$V = \{1,2\}$

1-Consider the image segment shown below.

(a) Let V = {0, 1} and compute D4-, D8- ja Dm-distances between p and q.

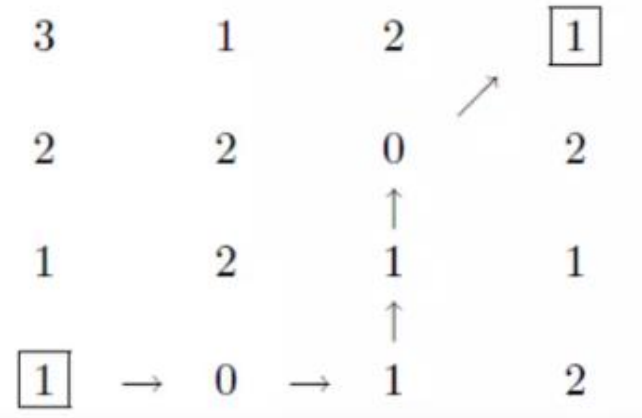
(b) Repeat for V = {1, 2}.

$$
\begin{array}{cccc}
3 & 1 & 2 & \boxed{1} \quad \leftarrow q \\
2 & 2 & 0 & 2 \\
1 & 2 & 1 & 1 \\
p \rightarrow \boxed{1} & 0 & 1 & 2
\end{array}
$$

a) D4(p, q) = |xp - xq| + |yp - yq| = 3 + 3 = 6

D8(p, q) = max(|xp - xq|, |yp - yq|) = max(3, 3) = 3

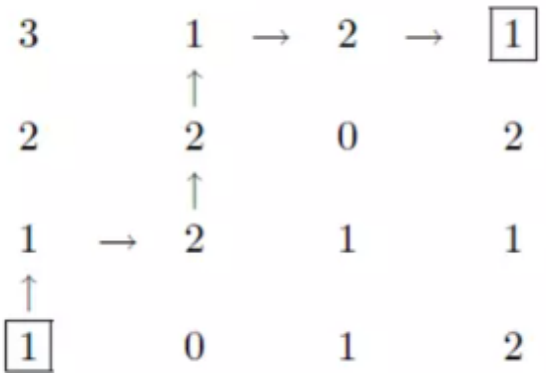
Dm(p, q) = 5



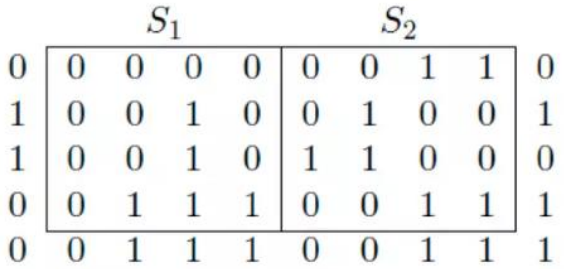
b) D4 and D8 are independent of V, so they are the same as in a)

Dm(p, q) = 6

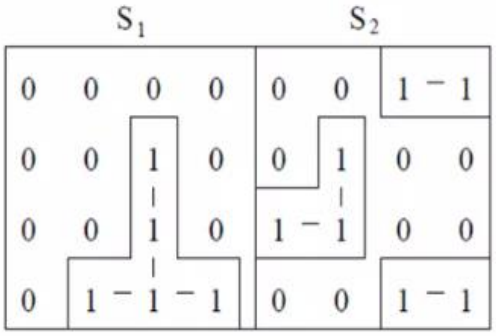
Of course, this is not the only possible path with length 6.



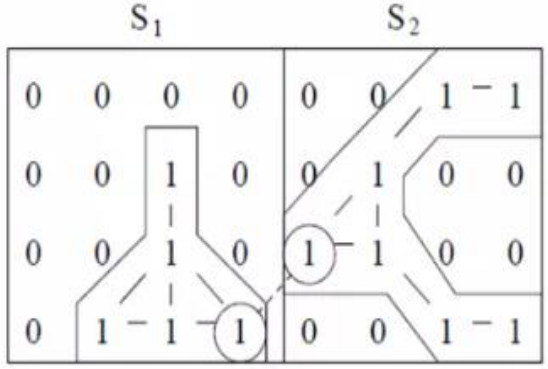
2-Consider the two image subsets S1 and S2 shown below. For V = {1}, determine how many
(a) 4-connected
(b) 8-connected
(c) m-connected
components there are in S1 and S2. Are S1 and S2 adjacent?



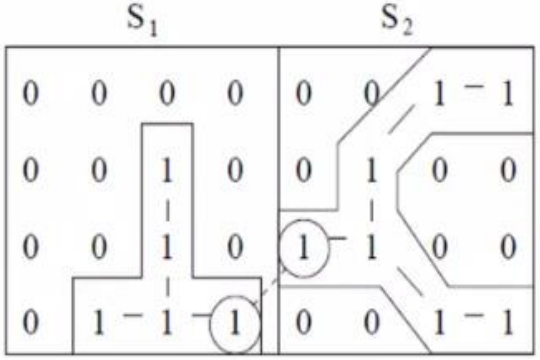
a) 4-connected components:
S1: 1, S2: 3



b) 8-connected components:
S1: 1, S2: 1



c) M-connected components:
S1: 1, S2: 1



Pixel Connectivity

V={1}

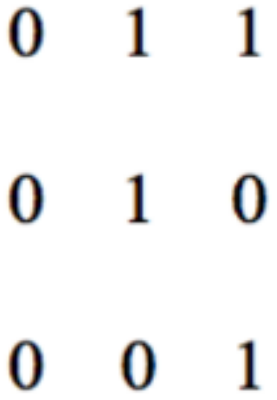


Fig: An arrangement of pixels

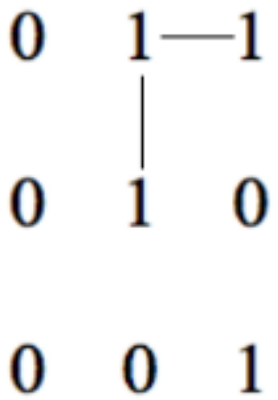


Fig: 4-connectivity of pixels

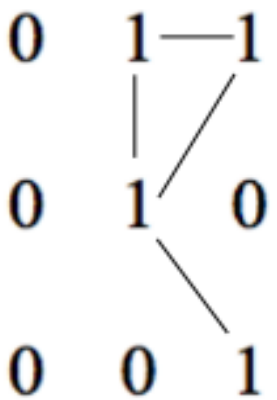


Fig: 8-connectivity of pixels

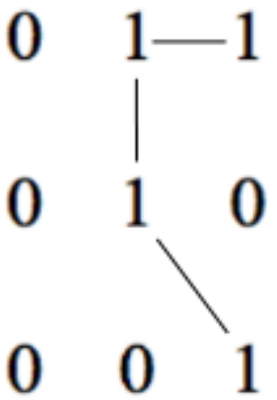


Fig: m-connectivity of pixels

Set of image locations of neighbors of a point p is called neighborhood of p.

Pixel Connectivity

***p*=(2,2) (center pixel with value 1)**

Given a binary image I (1 = foreground pixel, 0 = background):

$$I = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

4-connectivity neighbors of *p*:

Positions:

- Up: (1,2) → 1
- Down: (3,2) → 1
- Left: (2,1) → 1
- Right: (2,3) → 1

All neighbors have value 1

→ connected via 4-connectivity.

2 Connected components:
(1,2),(2,1),(2,2),(2,3),(3,2)
and
(3,4), (4,3), (4,4)

Pixel Connectivity

A binary image I (1 = foreground pixel, 0 = background):

$$I = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

p=(2,2) (center pixel with value 1)

8-connectivity neighbors of p:

4-connectivity Positions:

- Up: (1,2) → 1
- Down: (3,2) → 1
- Left: (2,1) → 1
- Right: (2,3) → 1

All neighbors have value 1
→ connected via 4-connectivity.

- Top-left: (1,1) → 0
- Top-right: (1,3) → 0
- Bottom-left: (3,1) → 0
- Bottom-right: (3,3) → 0

(2,3) and (3,4) → 1
(3,2) and (4,3) → 1
(3,4), (4,4) → 1
(4,3), (4,4) → 1

1 Connected components:
(1,2),(2,1),(2,2),(2,3),(3,2), (3,4), (4,3), (4,4)

Pixel Connectivity

A binary image I (1 = foreground pixel, 0 = background):

$$I = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

p=(2,2) (center pixel with value 1)

8-connectivity neighbors of p:

4-connectivity Positions:

- Up: (1,2) → 1
- Down: (3,2) → 1
- Left: (2,1) → 1
- Right: (2,3) → 1

All neighbors have value 1
→ connected via 4-connectivity.

+

(3,2) and (3,3) → 1
(3,2) and (4,3) → 1

1 Connected components:
(1,2),(2,1),(2,2),(2,3),(3,2), (4,3)

Pixel Connectivity

A binary image I (1 = foreground pixel, 0 = background):

$$I = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

p=(2,2) (center pixel with value 1)

m-connectivity neighbors of p:

4-connectivity Positions:

- Up: (1,2) → 1
- Down: (3,2) → 1
- Left: (2,1) → 1
- Right: (2,3) → 1

All neighbors have value 1
→ connected via 4-connectivity.

(3,2) and (4,3) → 1

1 Connected components:
(1,2),(2,1),(2,2),(2,3),(3,2), (4,3)

Pixel Connectivity Graph

Pixels with value 1 & their coordinates:
A: (1,2), B: (2,1), C: (2,2), D: (2,3), E: (3,2), F: (3,4), G: (4,3), H: (4,4)

Binary Image *I*:

$$I = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix} \Rightarrow \begin{bmatrix} 0 & A & 0 & 0 \\ B & C & D & 0 \\ 0 & E & 0 & F \\ 0 & 0 & G & H \end{bmatrix}$$

Edges for 4-Connectivity:

Node	4-Connected Neighbors
A (1,2)	C (2,2)
B (2,1)	C (2,2)
C (2,2)	A (1,2), B (2,1), D (2,3), E (3,2)
D (2,3)	C (2,2)
E (3,2)	C (2,2)
F (3,4)	H (4,4)
G (4,3)	H (4,4)
H (4,4)	F (3,4), G (4,3)

Graph edges:

A — C,
B — C,
C — D,
C — E,
F — H,
G — H

Pixel Connectivity Graph

Pixels with value 1 & their coordinates:
A: (1,2), B: (2,1), C: (2,2),
D: (2,3), E: (3,2), F: (3,4),
G: (4,3), H: (4,4)

Binary Image *I*:

$$I = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix} \rightarrow \begin{bmatrix} 0 & A & 0 & 0 \\ B & C & D & 0 \\ 0 & E & 0 & F \\ 0 & 0 & G & H \end{bmatrix}$$

Edges for 8-Connectivity:

Node	8-Connected Neighbors
D (2,3)	E (3,2), F (3,4)
E (3,2)	G (4,3)
F (3,4)	G (4,3)

Graph edges:
A — D,
D — E,
D — F,
E — G,
F — G

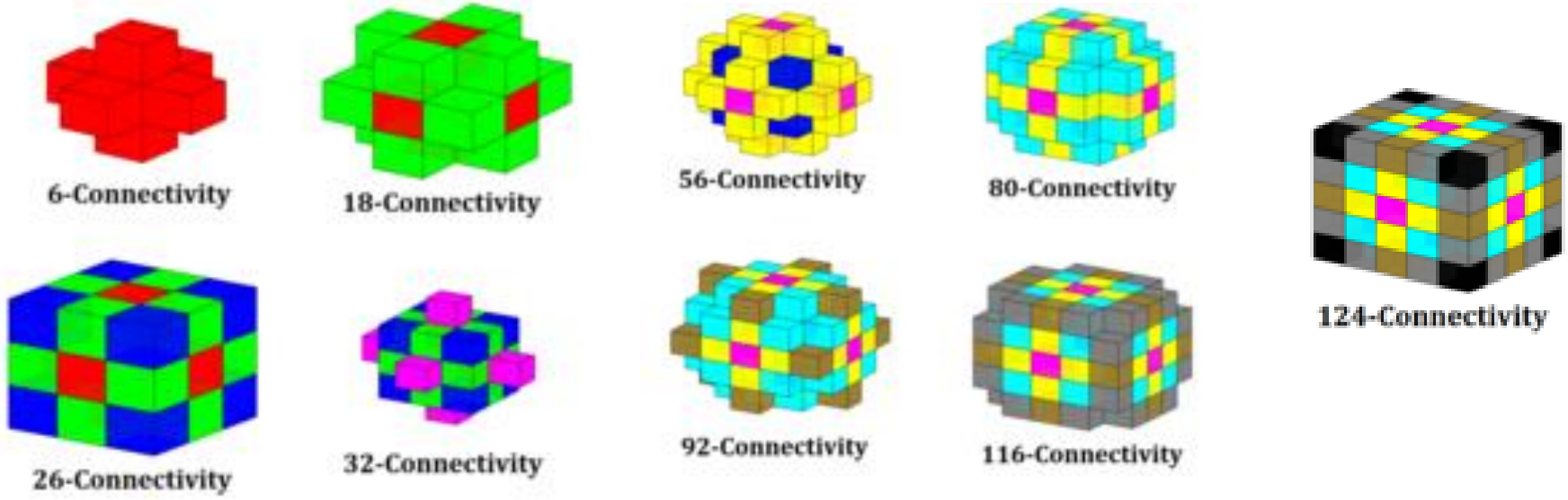
+

Graph edges:
A — C,
B — C,
C — D,
C — E,
F — H,
G — H

Pixel connectivity

V includes the center hypervoxel, which is not included in the connectivity. Subtracting 1 yields the neighborhood connectivity, G

$$G = V - 1$$



Pixel connectivity

Consider solving for $G|\vec{q} = (0,1,1)$

In this scenario, $N = 3$ since the vector is 3-dimensional. $n_0 = 1$ since there is one $q_i = 0$. Likewise, $n_1 = 2$. $k = 1, n = 3$ since $\max q_i = 1$. $d = \sqrt{0^2 + 1^2 + 1^2} = \sqrt{2}$. The neighborhood is M_3^3 and the hypersphere is $S_3^{\sqrt{2}}$

$$E = \frac{3!}{1! * 2! * 0!} 2^{3-1} = \frac{6}{2} 4 = 12$$

$$E = \frac{N!}{\prod_{j=0}^{k} n_j!} 2^{N-n_0}$$

The basic $\vec{q}$ in the neighborhood N_3^3, $\vec{q}_1 = (0,0,0)$. The Manhattan Distance between our vector and the basic vector is $\|\vec{q} - \vec{q}_0\|_1 = 2$, so $\vec{q} = \vec{q}_3$. Therefore,

$$G_{\vec{q}_3} = V_{\vec{q}_3} - 1 = E_{\vec{q}_1} + E_{\vec{q}_2} + E_{\vec{q}_3} - 1 = E_{\vec{q}=(0,0,0)} + E_{\vec{q}=(0,0,1)} + E_{\vec{q}=(0,1,1)}$$

$$E_{\vec{q}=(0,0,0)} = \frac{3!}{3! * 0! * 0!} 2^{3-3} = \frac{6}{6} 1 = 1$$

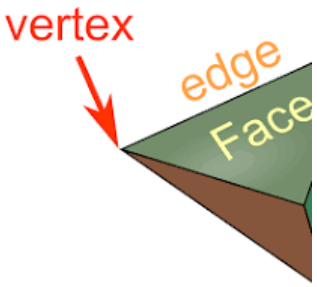
$$E_{\vec{q}=(0,0,1)} = \frac{3!}{2! * 1!} 2^{3-2} = \frac{6}{2} 2 = 6$$

$$G = 1 + 6 + 12 - 1 = 18$$

Which matches the supplied table

Table of Selected Connectivity

Neighborhood Size: M_N^n	Connectivity Type	Typical Vector: $\vec{q}$	Sphere Radius: d	Elements on Sphere: E	Elements in Sphere: V	Neighborhood Connectivity: G
1	edge	(0)	$\sqrt{0}$	1 × 1 = 1	1	0
3	point	(1)	$\sqrt{1}$	1 × 2 = 2	3	2
5	point-point	(2)	$\sqrt{4}$	1 × 2 = 2	5	4
...						
1 × 1	face	(0,0)	$\sqrt{0}$	1 × 1 = 1	1	0
3 × 3	edge	(0,1)	$\sqrt{1}$	2 × 2 = 4	5	4
	point	(1,1)	$\sqrt{2}$	1 × 4 = 4	9	8
5 × 5	edge-edge	(0,2)	$\sqrt{4}$	2 × 2 = 4	13	12
	point-edge	(1,2)	$\sqrt{5}$	2 × 4 = 8	21	20
	point-point	(2,2)	$\sqrt{8}$	1 × 4 = 4	25	24

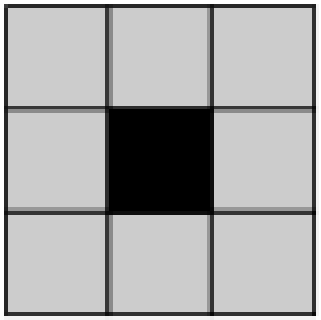


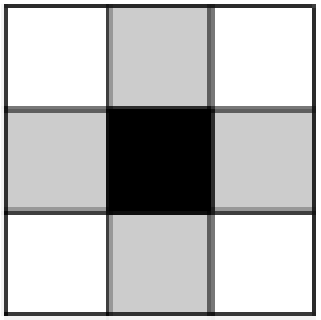
2-dimensional: 4-connected

4-connected pixels are neighbors to every pixel that touches one of their edges. These pixels are connected horizontally and vertically. In terms of pixel coordinates, every pixel that has the coordinates

$$(x \pm 1, y) \text{ or } (x, y \pm 1)$$

is connected to the pixel at (x, y).



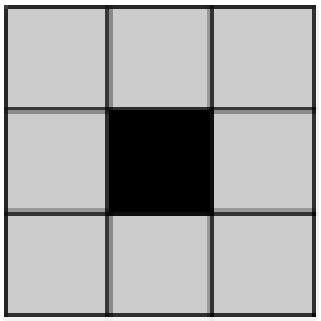


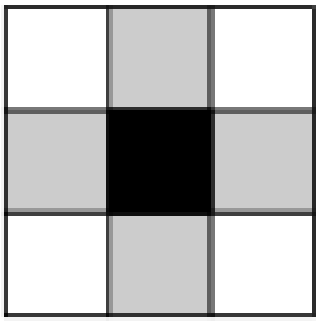
Example of neighborhood of pixels - association of eight and four pixels

2-dimensional: 6-connected

6-connected pixels are neighbors to every pixel that touches one of their corners (which includes pixels that touch one of their edges) in a hexagonal grid or stretcher bond rectangular grid.

There are several ways to map hexagonal tiles to integer pixel coordinates. With one method, in addition to the 4-connected pixels, the two pixels at coordinates $(x + 1, y + 1)$ and $(x - 1, y - 1)$ are connected to the pixel at (x, y).

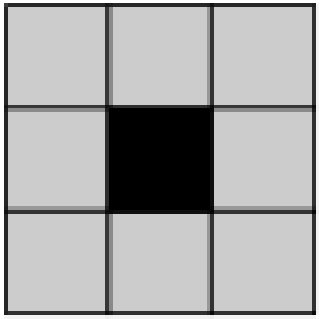


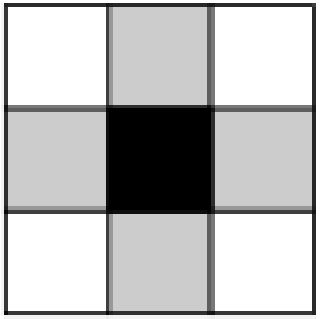


Example of neighborhood of pixels - association of eight and four pixels

2-dimensional: 8-connected

8-connected pixels are neighbors to every pixel that touches one of their edges or corners. These pixels are connected horizontally, vertically, and diagonally. In addition to 4-connected pixels, each pixel with coordinates $(x \pm 1, y \pm 1)$ is connected to the pixel at (x, y).





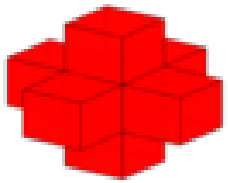
Example of neighborhood of pixels - association of eight and four pixels

Table of Selected Connectivity

1 × 1 × 1	volume	(0,0,0)	$\sqrt{0}$	1 × 1 = 1	1	0
3 × 3 × 3	face	(0,0,1)	$\sqrt{1}$	3 × 2 = 6	7	6
	edge	(0,1,1)	$\sqrt{2}$	3 × 4 = 12	19	18
	point	(1,1,1)	$\sqrt{3}$	1 × 8 = 8	27	26
5 × 5 × 5	face-face	(0,0,2)	$\sqrt{4}$	3 × 2 = 6	33	32
	edge-face	(0,1,2)	$\sqrt{5}$	6 × 4 = 24	57	56
	point-face	(1,1,2)	$\sqrt{6}$	3 × 8 = 24	81	80
	edge-edge	(0,2,2)	$\sqrt{8}$	3 × 4 = 12	93	92
	point-edge	(1,2,2)	$\sqrt{9}$	3 × 8 = 24	117	116
	point-point	(2,2,2)	$\sqrt{12}$	1 × 8 = 8	125	124

3-dimensional: 6-connected

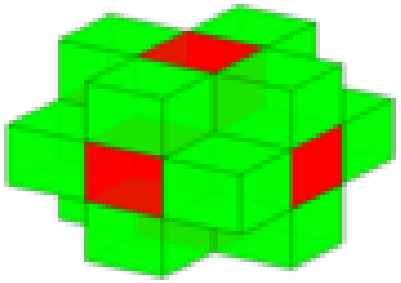
6-connected pixels are neighbors to every pixel that touches one of their faces. These pixels are connected along one of the primary axes. Each pixel with coordinates $(x \pm 1, y, z)$, $(x, y \pm 1, z)$, or $(x, y, z \pm 1)$ is connected to the pixel at (x, y, z).



6-Connectivity

3-dimensional: 18-connected

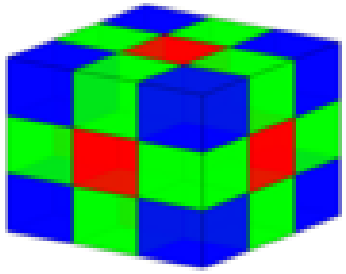
18-connected pixels are neighbors to every pixel that touches one of their faces or edges. These pixels are connected along either one or two of the primary axes. In addition to 6-connected pixels, each pixel with coordinates $(x \pm 1, y \pm 1, z)$, $(x \pm 1, y \mp 1, z)$, $(x \pm 1, y, z \pm 1)$, $(x \pm 1, y, z \mp 1)$, $(x, y \pm 1, z \pm 1)$, or $(x, y \pm 1, z \mp 1)$ is connected to the pixel at (x, y, z).



18-Connectivity

3-dimensional: 26-connected

26-connected pixels are neighbors to every pixel that touches one of their faces, edges, or corners. These pixels are connected along either one, two, or all three of the primary axes. In addition to 18-connected pixels, each pixel with coordinates $(x \pm 1, y \pm 1, z \pm 1)$, $(x \pm 1, y \pm 1, z \mp 1)$, $(x \pm 1, y \mp 1, z \pm 1)$, or $(x \mp 1, y \pm 1, z \pm 1)$ is connected to the pixel at (x, y, z).



26-Connectivity

Table of Selected Connectivity

$1 \times 1 \times 1 \times 1$	hypervolume	(0,0,0,0)	$\sqrt{0}$	$1 \times 1 = 1$	1	0
$3 \times 3 \times 3 \times 3$	volume	(0,0,0,1)	$\sqrt{1}$	$4 \times 2 = 8$	9	8
	face	(0,0,1,1)	$\sqrt{2}$	$6 \times 4 = 24$	33	32
	edge	(0,1,1,1)	$\sqrt{3}$	$4 \times 8 = 32$	65	64
	point	(1,1,1,1)	$\sqrt{4}$	$1 \times 16 = 16$	81	80

Thank You

Lecture 1.5

Digital Image Fundamentals

Linear and Nonlinear Operations, Simple Image Formation Model , Image Sampling and Quantization, K-Means Quantization

Instructor Name:

Professor Dr. Jahangir Alam

Professor, Dept. of CSE, BAUST

Linear and Nonlinear Operations

$$f = \begin{bmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \\ 70 & 80 & 90 \end{bmatrix}$$

Linear Operation: Image Smoothing with Box Filter

$$h = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$\begin{aligned} &Smoothed\ value \\ &= \frac{1}{9}(10 + 20 + 30 + 40 + 50 + 60 + 70 + 80 + 90) \\ &= 450/9 = 50 \end{aligned}$$

Result is a weighted sum → satisfies linearity.

Linear and Nonlinear Operations

$$f = \begin{bmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \\ 70 & 80 & 90 \end{bmatrix}$$

All pixel values flattened:
{10,20,30,40,50,60,70,80,90}

➡ Median = 50

Now change pixel (center value = 50) to 500:

$$New\ Neighborhood = \begin{bmatrix} 10 & 20 & 30 \\ 40 & 500 & 60 \\ 70 & 80 & 90 \end{bmatrix}$$

Sorted: {10,20,30,40,60,70,80,90,500} ➡ Median = 60

(different from 50)

Median does not scale linearly → so, median filter is nonlinear.

Simple Image Formation Model

2D : f(x,y)
nonnegative
and finite

$$0 \le f(x,y) \le \infty$$

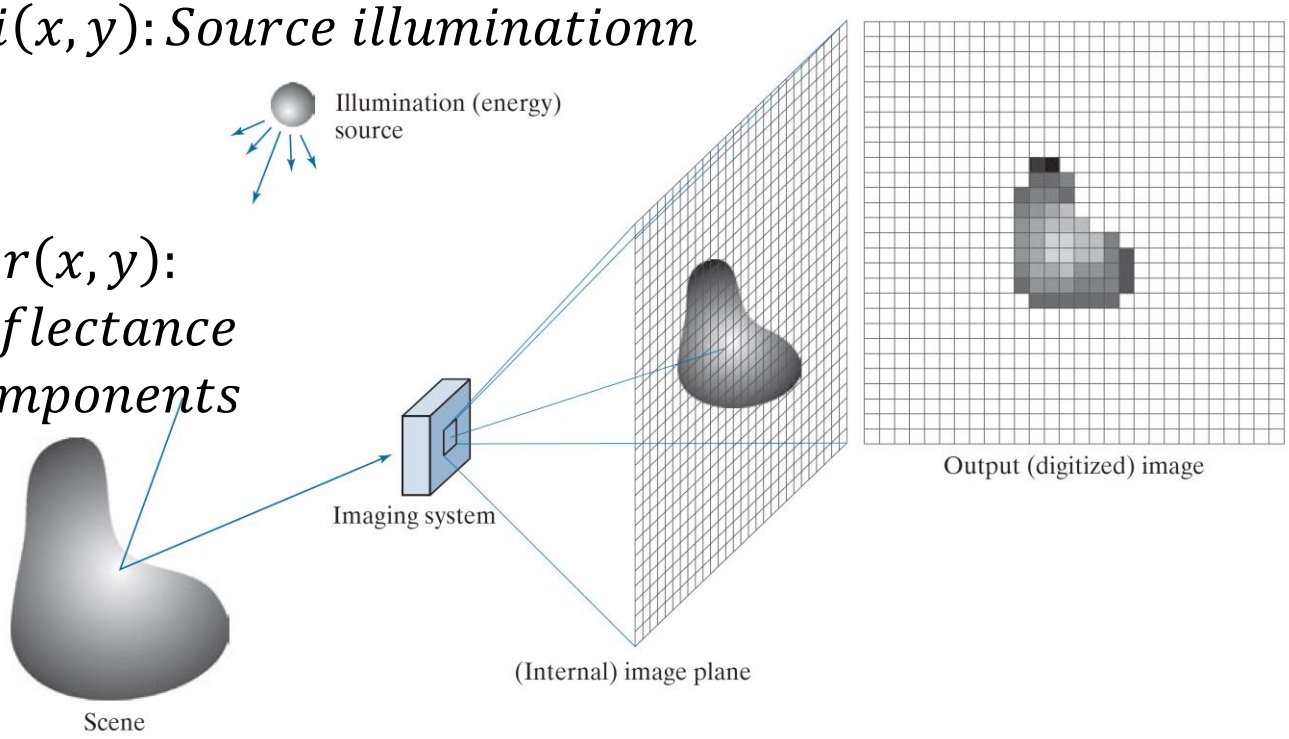
$$f(x,y) = i(x,y)r(x,y)$$

$$0 \le i(x,y) < \infty$$

$$0 \le r(x,y) \le 1$$

$i(x,y)$: *Source illuminationn*

$r(x,y)$: *Reflectance components*



Simple Image Formation Model

$$f(x,y) = i(x,y)r(x,y)$$

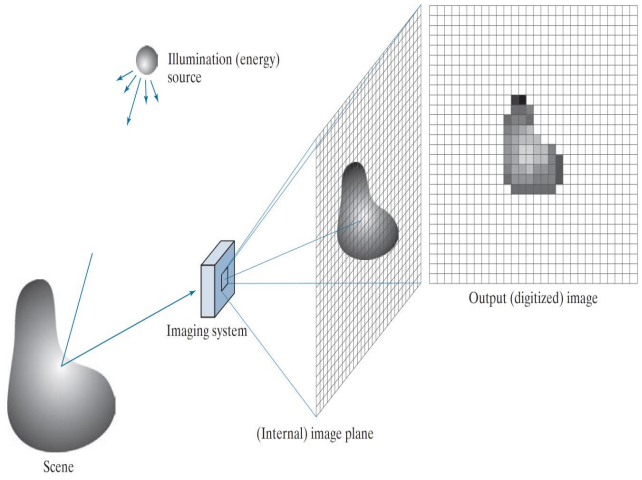
$$0 \le i(x,y) < \infty \qquad 0 \le r(x,y) \le 1$$

Illumination source

Total absorption

Characteristics of imaged objects

Total reflectance



Simple Image Formation Model

Intensity (gray level) of a monochrome image at any coordinates (x, y):

$$l = f(x, y)$$

$$L_{min} \leq l \leq L_{max}$$

$$L_{min} = i_{min}\, r_{min}$$

$$L_{max} = i_{max}\, r_{max}$$

Intensity (Gray) scale interval $[L_{min}, L_{max}]$ → $[0,1]$ or $[0, C]$

Black White

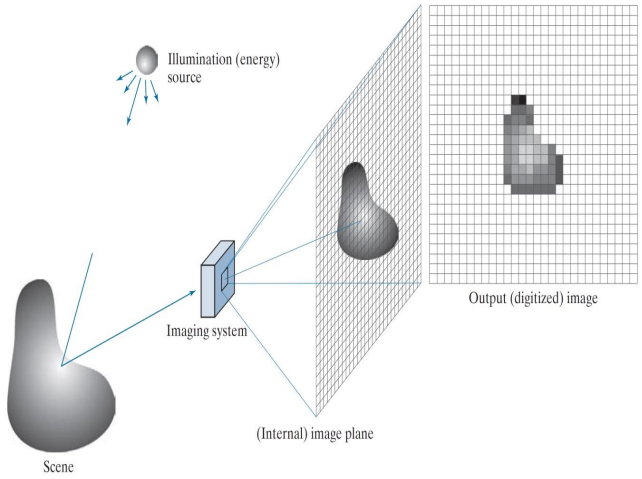
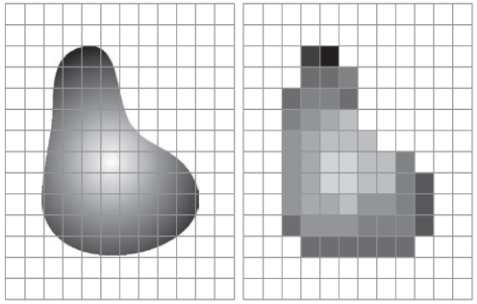
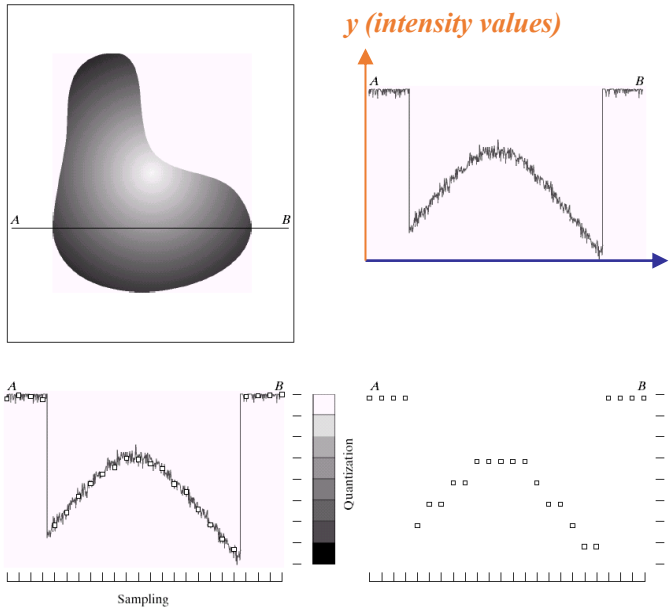


Image Sampling and Quantization

$Digitization\ of\ f(x,y)$

$Sampling$ $(Amplitude)$ $Quantization$

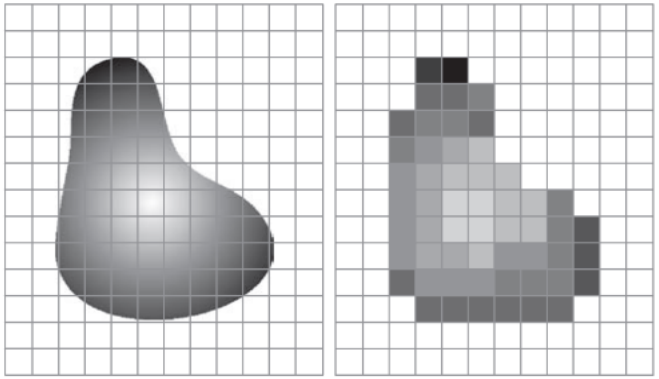


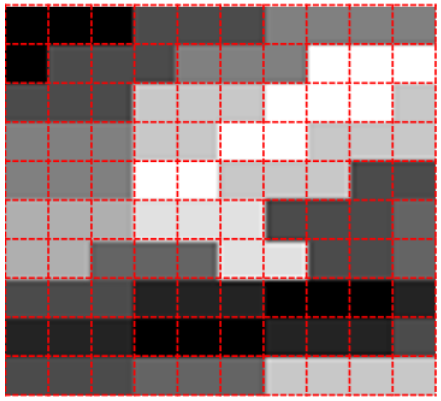


(a) Continuous image projected onto a sensor array.

(b) Result of image sampling and quantization.

Image Sampling and Quantization





0	0	0	75	75	75	128	128	128	128
0	75	75	75	128	128	128	255	255	255
75	75	75	200	200	200	255	255	255	200
128	128	128	200	200	255	255	200	200	200
128	128	128	255	255	200	200	200	75	75
175	175	175	225	225	225	75	75	75	100
175	175	100	100	100	225	225	75	75	100
75	75	75	35	35	35	0	0	0	35
35	35	35	0	0	0	35	35	35	75
75	75	75	100	100	100	200	200	200	200

Image Sampling and Quantization

Sampling



1024



512



256



128



64



32

```
im=imread('obelix.jpg');
im=rgb2gray(imread('obelix.jpg'));
im1=imresize(im, [1024 1024]);
im2=imresize(im1, [1024 1024]/2);
im3=imresize(im1, [1024 1024]/4);
im4=imresize(im1, [1024 1024]/8);
im5=imresize(im1, [1024
1024]/16);
im6=imresize(im1, [1024
1024]/32);

figure;imshow(im1)
figure;imshow(im2)
figure;imshow(im3)
figure;imshow(im4)
figure;imshow(im5)
```

Image Sampling and Quantization

Sampling

```
figure;
subplot(2,3,1);
imshow(im1);
subplot(2,3,2);
imshow(im2)
subplot(2,3,3);
imshow(im3);
subplot(2,3,4);
imshow(im4)
subplot(2,3,5);
imshow(im5);
subplot(2,3,6);
imshow(im6)
```



1024



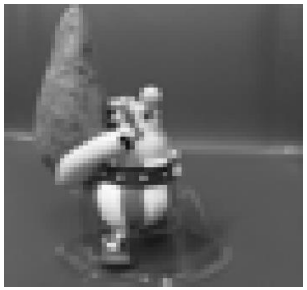
512



256



128



64



32

Quantization

```
im=imread('obelix.jpg');
im=rgb2gray(imread('obelix.jpg'));
im1=imresize(im, [1024 1024]);
im2= gray2ind(im1,2^7);
im3= gray2ind(im1,2^6);
im4= gray2ind(im1,2^5);
im5= gray2ind(im1,2^4);
im6= gray2ind(im1,2^3);
im7= gray2ind(im1,2^2);
im8= gray2ind(im1,2^1);
figure;
subplot(2,4,1);imshow(im1,[]);subplot(2,4,2);imshow(im2,[])
subplot(2,4,3);imshow(im3,[]);subplot(2,4,4);imshow(im4,[])
subplot(2,4,5);imshow(im5,[]);subplot(2,4,6);imshow(im6,[])
subplot(2,4,7);imshow(im7,[]);subplot(2,4,8);imshow(im8,[])
```

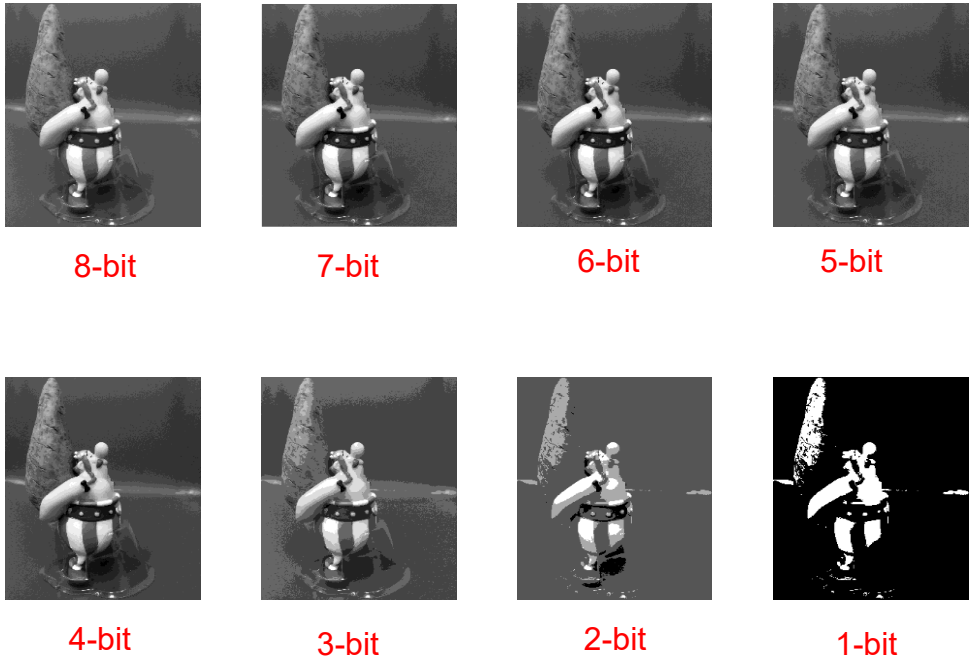


Image Sampling and Quantization

$$\begin{bmatrix} 100 & 105 & 110 & 115 & 120 & 125 \\ 130 & 135 & 140 & 145 & 150 & 155 \\ 160 & 165 & 170 & 175 & 180 & 185 \\ 190 & 195 & 200 & 205 & 210 & 215 \\ 220 & 225 & 230 & 235 & 240 & 245 \\ 250 & 255 & 240 & 225 & 210 & 195 \end{bmatrix}$$

Sampling Down to 3×3

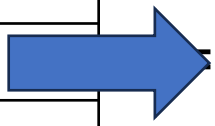
We'll reduce the image size by a **factor of 2**, picking every **2nd row and 2nd column** (nearest-neighbor sampling):

Pick these positions (original indices are shown for clarity):

Original Position	Pixel Value
(0,0)	100
(0,2)	110
(0,4)	120
(2,0)	160
(2,2)	170
(2,4)	180
(4,0)	220
(4,2)	230
(4,4)	240

Image Sampling and Quantization

Original Position	Pixel Value
(0,0)	100
(0,2)	110
(0,4)	120
(2,0)	160
(2,2)	170
(2,4)	180
(4,0)	220
(4,2)	230
(4,4)	240



$$Sampled\ Image = \begin{bmatrix} 100 & 110 & 120 \\ 160 & 170 & 180 \\ 220 & 230 & 240 \end{bmatrix}$$

Concept	What You Did
Sampling	Picked every 2nd row and column
Resolution Reduced	From 6×6 → 3×3 pixels
Information Loss	Detail and smoothness reduced
	No code or software needed

Image Sampling and Quantization

$$Original\ Image\ (8-bit) = \begin{bmatrix} 34 & 80 & 120 & 200 \\ 50 & 90 & 130 & 210 \\ 60 & 100 & 140 & 220 \\ 70 & 110 & 150 & 230 \end{bmatrix}$$

Quantization (Reduce to 3-bit levels)

- 8-bit grayscale → 256 levels (0–255)
- 3-bit grayscale → $2^3 = 8$ levels: 0 to 7
- **Quantization Step** = $256/8 = 32$

Image Sampling and Quantization

$$Original\ Image\ (8-bit) = \begin{bmatrix} 34 & 80 & 120 & 200 \\ 50 & 90 & 130 & 210 \\ 60 & 100 & 140 & 220 \\ 70 & 110 & 150 & 230 \end{bmatrix}$$

- **Quantization**

  Step $= 256/$

  $8 = 32$

We divide each original value by 32 and **take the integer part**:

$$Quantized\ Image\ (3-bit) = \begin{bmatrix} 1 & 2 & 3 & 6 \\ 1 & 2 & 4 & 6 \\ 1 & 3 & 4 & 6 \\ 2 & 3 & 4 & 7 \end{bmatrix}$$

Image Quantization

$$\begin{bmatrix} 23 & 85 & 140 \\ 210 & 173 & 60 \\ 45 & 90 & 250 \end{bmatrix}$$

Objective: Reduce the image to **3-bit quantization** (i.e., 8 gray levels).

Determine quantization levels

3 bits $\rightarrow$

$2^3 = 8$ levels.

- Intensity range:

  0 to 255.

- Each quantization level covers a range of: $256/8 = 32$

Image Quantization

- The quantized image has only 8 intensity levels instead of 256.

Level	Intensity Range	Quantized Value (representative)
0	0 – 31	15 (midpoint)
1	32 – 63	47
2	64 – 95	79
3	96 – 127	111
4	128 – 159	143
5	160 – 191	175
6	192 – 223	207
7	224 – 255	239

$$\begin{bmatrix} 23 & 85 & 140 \\ 210 & 173 & 60 \\ 45 & 90 & 250 \end{bmatrix}$$

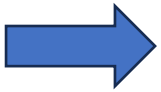


Quantized Image (3-bit)

$$\begin{bmatrix} 15 & 79 & 143 \\ 207 & 175 & 47 \\ 47 & 79 & 239 \end{bmatrix}$$

Color Quantization or Quantization Color Image

Pixel	R	G	B
1	123	200	90
2	130	210	85
3	140	220	80



Level (2-bit)	Range
00	0–63
01	64–127
10	128–191
11	192–255

Pixel	Original RGB	Quantized (bin)	Quantized RGB
1	(123, 200, 90)	01, 11, 01	(96, 224, 96)
2	(130, 210, 85)	10, 11, 01	(160, 224, 96)
3	(140, 220, 80)	10, 11, 01	(160, 224, 96)

The color image is now approximated with only 3 distinct RGB values, reducing memory use and allowing compression.

Thank You

Lecture 1.6

Digital Image Fundamentals

K-Means Quantization, Image Interpolation

Instructor Name:

Professor Dr. Jahangir Alam

Professor, Dept. of CSE, BAUST

K-means-based or Grayscale Quantization

Gray levels: X=[34, 96, 123, 187, 222]

Quantize to 4 levels (2-bit):

Level (binary)	Value range	Midpoint
00	0–63	32
01	64–127	96
10	128–191	160
11	192–255	224

Pixel	Range	Quantized Level	New Value
34	0–63	00	32
96	64–127	01	96
123	64–127	01	96
187	128–191	10	160
222	192–255	11	224

X=[32, 96, 96, 160, 224]

This reduces file size and simplifies processing

K-Means Quantization (on Grayscale)

Gray levels: X=[34, 96, 123, 187, 222]

Let's choose K = 2 (2 clusters).

Let's initial centroids:
C1 = 96
C2 = 222

Assign Pixels to Closest Cluster

Pixel	Dist to 96	Dist to 222	Cluster
34	62	188	C1
96	0	126	C1
123	27	99	C1
187	91	35	C2
222	126	0	C2

Clusters: C1: [34, 96, 123]; C2: [187, 222]

K-Means Quantization (on Grayscale)

Clusters: C1: [34, 96, 123]; C2: [187, 222]

Update Centroids

C1=mean([34, 96, 123]) = 253/3 ≈ 84.3

C2 = mean([187, 222]) = 409/2 = 204.5

Assign Pixels to Closest Cluster:

Pixel	Dist to 84.3	Dist to 204.5	New Cluster
34	50.3	170.5	C1
96	11.7	108.5	C1
123	38.7	81.5	C1
187	102.7	17.5	C2
222	137.7	17.5	C2

Same Clusters:
C1: [34, 96, 123];
C2: [187, 222]

Quantization: Max-Lloyd Algorithm

1. Choose initial quantization levels;
2. Assign points to a quantization level and reconstruct image;
3. Compute the new quantization levels as the mean of the value of all points assigned to each quantization level.
4. Go back to 2 until reduction of MSE is minimal.

$$\hat{x}_q \quad q = 0, 1, 2, \ldots, M-1$$

$$t_q = \frac{1}{2}\left(\hat{x}_{q-1} + \hat{x}_q\right) \quad q = 1, 2, \ldots, M-1$$

$$\hat{x}_q = \frac{\int_{t_q}^{t_{q+1}} x \cdot f_X(x)dx}{\int_{t_q}^{t_{q+1}} f_X(x)dx} \quad q = 0, 1, \ldots, M-1$$

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}(x_i - \hat{x}_i)^2,$$

Quantization: Max-Lloyd Algorithm

$$X = \{12, 15, 22, 29, 35, 42, 48, 52\}$$

Objective: Quantize X into **2 levels ($K = 2$)** using Max-Lloyd algorithm.

Choose initial quantization levels (e.g., random or evenly spread):

$$q_1^{(0)} = 15, q_2^{(0)} = 45$$

Group 1: samples $\leq 30 \leq 30$: $\{12,15,22,29\}$

Group 2: samples $> 30 > 30$: $\{35,42,48,52\}$

Quantization: Max-Lloyd Algorithm

Group 1: samples $\leq 30 \leq 30$: {12,15,22,29}
Group 2: samples $> 30 > 30$: {35,42,48,52}

Update quantization levels as centroids (means)

$$q_1^{(1)} = mean\ of\ group\ 1 = \frac{12 + 15 + 22 + 29}{4} = 19.5$$

$$q_2^{(1)} = mean\ of\ group\ 2 = \frac{35 + 42 + 48 + 52}{4} = 44.25$$

Recompute decision boundary, $b^{(1)} = \frac{19.5+44.25}{3} = 31.875$

Re-partition samples using new boundary $b^{(1)}$

- Group 1: $\leq 31.875 \leq 31.875$: {12,15,22,29}} (same as before)
- Group 2: $> 31.875 > 31.875$: {35,42,48,52} (same as before)

Quantization: Max-Lloyd Algorithm

Update quantization levels again

$$q_1^{(2)} = 19.5, q_2^{(2)} = 44.25$$

Final Result:

- Quantization levels: 19.5 and 44.25
- Decision boundary: 31.875

Sample	Quantized to
12	19.5
15	19.5
22	19.5
29	19.5
35	44.25
42	44.25
48	44.25
52	44.25

Image Interpolation

$$I = \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix}$$

Objective: Upsample (resize) this image to **4×4** using interpolation.

Nearest Neighbor (KNN) Interpolation:

Calculate scale factor

- From 2×2 to 4×4 → scale factor = 2 (both width and height)

Map new pixels to original pixels

- Each pixel in new image corresponds to the nearest original pixel.

Image Interpolation: Sampling Resolution

- The main idea is to use interpolation.

zooming, shrinking, rotating, and geometrically correcting digital images.

- Common methods are:

✓ Nearest neighbor interpolation

✓ Bilinear interpolation (4 nearest neighbor)

$$v(x, y) = ax + by + cxy + d$$

✓ Bicubic interpolation (16 nearest neighbor)

$$v(x, y) = \sum_{i=0}^{3} \sum_{j=0}^{3} a_{ij} x^i y^j$$



2D Bicubic Interpolation on Color Image

R-channel values (example):

$$\begin{bmatrix} 60 & 70 & 80 & 90 \\ 65 & 75 & 85 & 95 \\ 70 & 80 & 90 & 100 \\ 75 & 85 & 95 & 105 \end{bmatrix}$$

We want to compute R at position (1.5, 1.5)?

Interpolate along x (for each row at $y = 0\ to\ 3$)
Using Catmull-Rom cubic interpolation for each row:

$$C(t)$$
$$= 0.5 \times [2P_0 + (-P_{-1} + P_1)t + (2P_{-1} - 5P_0 + 4P_1 - P_2)t^2 + (-P_{-1} + 3P_0 - 3P_1 + P_2)t^3]$$

Let's interpolate between column positions 0–3 with $x = 0.5$

2D Bicubic Interpolation on Color Image

R-channel values (example):

$$\begin{bmatrix} 60 & 70 & 80 & 90 \\ 65 & 75 & 85 & 95 \\ 70 & 80 & 90 & 100 \\ 75 & 85 & 95 & 105 \end{bmatrix}$$

Row 0: $[60\ 70\ 80\ 90] \rightarrow\ f_0 =\ 60, f_1 =\ 70, f_2 =\ 80, f_3 =\ 90,$

$$C(t) = 0.5 \times [2P_0 + (-P_{-1} + P_1)t + (2P_{-1} - 5P_0 + 4P_1 - P_2)t^2 + (-P_{-1} + 3P_0 - 3P_1 + P_2)t^3]$$

$$f_0(x = 0.5) = 0.5\times (2 \times 70 + (-60 + 80)(0.5)^1 + (2 \times 60 - 5 \times 70 + 4 \times 80 - 90)(0.5)^2 + (-60 + 3 \times 70 - 3 \times 80 + 90)(0.5)^3) \approx 75$$

2D Bicubic Interpolation on Color Image

R-channel values (example):

$$\begin{bmatrix} 60 & 70 & 80 & 90 \\ 65 & 75 & 85 & 95 \\ 70 & 80 & 90 & 100 \\ 75 & 85 & 95 & 105 \end{bmatrix}$$

Repeat similarly for rows 1, 2, 3 → get intermediate values:

Row1 → 80
Row2 → 85
Row3 → 90

2D Bicubic Interpolation on Color Image

R-channel values (example):

$$\begin{bmatrix} 60 & 70 & 80 & \mathbf{90} \\ 65 & 75 & \mathbf{85} & 95 \\ 70 & \mathbf{80} & 90 & 100 \\ \mathbf{75} & 85 & 95 & 105 \end{bmatrix}$$

Interpolate along y using the 4 intermediate values

Now interpolate:

$$[Row0, Row1, Row2, Row3] = [75, 80, 85, 90] \; at \; y = 0.5$$

Apply same cubic interpolation:

$$f(0.5)$$
$$= 0.5$$
$$\times (2 \times 80 + (85 - 75)(0.5)$$
$$+ (2 \times 75 - 5 \times 80 + 4 \times 85 - 90)(0.5)^2$$
$$+ (-75 + 3 \times 80 - 3 \times 85 + 90)(0.5)^3) \approx 82.5$$

2D Bicubic Interpolation on Color Image

R-channel values (example):

$$\begin{bmatrix} 60 & 70 & 80 & 90 \\ 65 & 75 & 85 & 95 \\ 70 & 80 & 90 & 100 \\ 75 & 85 & 95 & 105 \end{bmatrix}$$

Final Interpolated *R Value* ≈ *82.5 at (1.5, 1.5)*
You would repeat the same process for G and B channels.

Channel	Result at (1.5, 1.5)
R	82.5
G	≈ (compute similarly)
B	≈ (compute similarly)

Image Interpolation

$$I = \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix}$$

Objective: Upsample (resize) this image to **4×4** using interpolation.

Nearest Neighbor (KNN) Interpolation:

Calculate scale factor

- From 2×2 to 4×4 → scale factor = 2 (both width and height)

Map new pixels to original pixels

- Each pixel in new image corresponds to the nearest original pixel.

Image Interpolation using KNN

New Position	Corresponding Original Pixel	Value
(1,1)	(1,1)	10
(1,2)	(1,1)	10
(1,3)	(1,2)	20
(1,4)	(1,2)	20
(2,1)	(1,1)	10
(2,2)	(1,1)	10
(2,3)	(1,2)	20
(2,4)	(1,2)	20
(3,1)	(2,1)	30
(3,2)	(2,1)	30
(3,3)	(2,2)	40
(3,4)	(2,2)	40
(4,1)	(2,1)	30
(4,2)	(2,1)	30
(4,3)	(2,2)	40
(4,4)	(2,2)	40

$$I = \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix}$$

Resulting 4×4 image:

$$\begin{bmatrix} 10 & 10 & 20 & 20 \\ 10 & 10 & 20 & 20 \\ 30 & 30 & 40 & 40 \\ 30 & 30 & 40 & 40 \end{bmatrix}$$

Image Interpolation (Bilinear Interpolation)

Bilinear Interpolation:

Original Pixel	Value
(1,1)	10
(1,2)	20
(2,1)	30
(2,2)	40

$$I = \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix}$$

We want to find pixel values at intermediate points in the 4×4 grid.

$$f(x,y) = f_{11}(1-a)(1-b) + f_{21}a(1-b) + f_{12}(1-a)b + f_{22}ab$$

Where: $a = x - 1, b = y - 1$

Image Interpolation (Bilinear Interpolation)

$$I = \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix}$$

Original Pixel	Value
(1,1)	10
(1,2)	20
(2,1)	30
(2,2)	40

Compute some intermediate pixel values

- At $(1.5, 1.5)$: $a = 0.5, b = 0.5$

$$f(1.5,1.5) = 10 \times 0.5 \times 0.5 + 30 \times 0.5 \times 0.5 + 20 \times 0.5 \times 0.5 + 40 \times 0.5 \times 0.5 = 25$$

- At $(1, 1.5)$: $a = 0, b = 0.5$

$$f(1,1.5) = 10 \times 1 \times 0.5 + 30 \times 0 \times 0.5 + 20 \times 1 \times 0.5 + 40 \times 0 \times 0.5 = 15$$

Image Interpolation (Bilinear Interpolation)

$$I = \begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix}$$

New Position	Corresponding Original Pixel	Value
(1,1)	(1,1)	10
(1,2)	(1,1.5)	15
(1,3)	(1,2)	20
(1,4)	(1,2)	20
(2,1)	(1,1.5)	20
(2,2)	(1.5,1.5)	25
(2,3)	(1.5,2)	30
(2,4)	(2,2.5)	30
(3,1)	(2,1)	30
(3,2)	(2,1.5)	35
(3,3)	(2,2)	40
(3,4)	(2,2.5)	40
(4,1)	(2.5,1)	30
(4,2)	(2,1.2)	30
(4,3)	(2,2)	40
(4,4)	(2,2.5)	40

Original Pixel	Value
(1,1)	10
(1,2)	20
(2,1)	30
(2,2)	40

3×3 image:

$$\begin{bmatrix} 10 & 15 & 20 \\ 20 & 25 & 30 \\ 30 & 35 & 40 \end{bmatrix}$$

Resulting 4×4 image:

$$\begin{bmatrix} 10 & 15 & 20 & 20 \\ 20 & 25 & 30 & 30 \\ 30 & 35 & 40 & 40 \\ 30 & 30 & 40 & 40 \end{bmatrix}$$

X-ray Image Analysis to Detect Abnormal Regions

100	100	105	100	100
100	90	80	90	100
105	80	**50**	80	105
100	90	80	90	100
100	100	105	100	100

Low values (like 50) can indicate a fracture or abnormality (dark spot).

Apply threshold to detect dark abnormal regions

Choose threshold T=70:

"1" indicates potential abnormal (dark) region.

0	0	0	0	0
0	0	1	0	0
0	1	1	1	0
0	0	1	0	0
0	0	0	0	0

X-ray Image Analysis to Detect Abnormal Regions

Step	Operation	Result
Thresholding	Detect dark pixels	Binary image highlighting dark regions
Connected components	Group neighboring pixels	Identify suspicious abnormal region
Region analysis	Measure size and shape	Confirm presence of fracture

X-ray Image Analysis to Detect Abnormal Regions

0	0	0	0	0
0	0	1	0	0
0	1	1	1	0
0	0	1	0	0
0	0	0	0	0

Connected component labeling to find the abnormal region

The cluster of 1's in the middle forms a connected region.

Calculate region properties (size)

- No. of pixels in abnormal region = 5
- Small connected region indicates possible fracture spot.

Step	Operation	Result
Thresholding	Detect dark pixels	Binary image highlighting dark regions
Connected components	Group neighboring pixels	Identify suspicious abnormal region
Region analysis	Measure size and shape	Confirm presence of fracture

Thank You